

Development of Smart Eye-Detecting Cars in Washing Area

Md. Abdullah Ibne Aziz

ID# 22203246

A Practicum in the Partial Fulfillment of the Requirements
for the Award of Bachelor of Computer Science and Engineering (BCSE)



Department of Computer Science and Engineering
College of Engineering and Technology
IUBAT—International University of Business Agriculture and Technology

Spring 2026

Development of Smart Eye-Detecting Cars in Washing Area

Md. Abdullah Ibne Aziz

A Practicum in the Partial Fulfillment of the Requirements for the Award of Bachelor of
Computer Science and Engineering (BCSE)

The practicum has been examined and approved,

Prof. Dr. Utpal Kanti Das
Professor & Chair, Dept of CSE
Dean, ISCSE

Shahinur Alam
Co-supervisor, Coordinator and Assistant Professor

Md. Khairul Islam
Supervisor and Lecturer

Department of Computer Science and Engineering
College of Engineering and Technology
IUBAT—International University of Business Agriculture and Technology

Spring 2026

Letter of Transmittal

12 May 2026

The Chair

Practicum Defense Committee

Department of Computer Science and Engineering

IUBAT—International University of Business Agriculture and Technology

4 Embankment Drive Road, Sector 10, Uttara Model Town

Dhaka 1230, Bangladesh.

Subject: Letter of Transmittal.

Dear Sir,

With due respect, I would like to inform you that it is a great pleasure for me to submit this report entitled “Development of Smart Eye-Detecting Cars in Washing Area” for the fulfillment of my practicum course. It was a great opportunity for me to work on this project to actualize my theoretical knowledge more realistically, and I gained a lot of exposure to the corporate culture of a renowned company. Now I am looking forward to your kind commentary on this practicum report.

I shall always be very grateful to you if you kindly go through this report and evaluate my performance.

Yours sincerely,

Md. Abdullah Ibne Aziz

ID #22203246

Organization's Certificate

Organizational Certificate



Date: 10 May, 2026

This is to certify that Md. Abdullah Ibne Aziz, who is pursuing B.Sc. in Computer Science from International University of Business, Agriculture and Technology, Dhaka, did his internship from 01 February 2026 and expected end date is 31st May 2026 (40 hrs/week).

As a Jr. Artificial Intelligence Engineer (Intern), his major responsibilities were:

- Developing and training AI/ML models.
- Data preprocessing and dataset management.
- Model evaluation and performance optimization.
- Assisting in computer vision related tasks.
- Collaboration with the development team for AI integration and testing.

Warm Regards,

MD Shaiful Islam Bapin
Chief Executive Officer, AIQNIC

📍 House-B177, Road-11,
Jahangirnagar co-operative
housing society
Savar, Dhaka-1343
☎ 01677386036
✉ contact@aiqnic.com

Student's Declaration

I, Md. Abdullah Ibne Aziz with ID# 22203246, am a student of BCSE (Bachelor of Computer Science and Engineering) program from the IUBAT School of Computer Science and Engineering (ISCSE) at the IUBAT - International University of Business Agriculture and Technology. I declare that this report entitled “Development of Smart Eye-Detecting Cars in Washing Area” has been prepared for the completion of the internship course, “CSC 490 Practicum” and part of the requirement for the award of the degree of Bachelor of Computer Science and Engineering (BCSE).

This report and project are solely my work. The modules and process of this project were designed from detailed research and analysis from several resources and references found on the internet. This work is not being, nor has been, submitted for any academic award, publication or presentation elsewhere.

Md. Abdullah Ibne Aziz
ID #22203246

Supervisor's Certification

This is to certify that the practicum report entitled “Development of Smart Eye-Detecting Cars in Washing Area” has been completed successfully by Md. A partial fulfillment of requirements for practicum defense course by Md. Abdullah Ibne Aziz (ID# 22203246) from the International University of Business Agriculture and Technology (IUBAT).

This report has been compiled and written under my supervision and guidance and is an overview of the work successfully accomplished by the student. As far as I am aware and as the student has declared no portion of this report has been submitted elsewhere for any degree, diploma or certificate.

For this reason, I believe this report should be submitted and I wish him the best in all future endeavors.

Md. Khairul Islam

Supervisor and Lecturer

Department of Computer Science and Engineering

IUBAT—International University of Business Agriculture and Technology

Abstract

This practicum report outlines the development of "Smart Eye" an AI-based Vehicle Monitoring and Session Management System for the car wash industry. The system was designed to automate all the tasks of vehicle detection, tracking, session logging, and occupancy monitoring, which are usually done manually and can be inefficient and prone to errors.

Smart Eye leverages a client-provided CCTV video to train a custom YOLO11 object detection model (Redmon et al., 2016; Jocher, Chaurasia and Qiu, 2024) that allows it to detect vehicles accurately in real time. The system is equipped with the ByteTrack multi-object tracking algorithm (Zhang et al., 2022), and a custom Re-Identification (Re-ID) module (Ye et al., 2022) to ensure stable vehicle identities during monitoring. The desktop application was built using Python and PyQt5 to allow real-time monitoring, session history management, visualization of analytics and configurable alert systems. Vehicle session records, timestamps and snapshots captured are stored in an SQLite database (Riverbed Technology, no date).

The project was created following the Incremental Process Model (Pressman and Maxim, 2014; Sommerville, 2016) as it allowed for the implementation and testing of each individual system element. The final product showcases the potential of computer vision and AI in enhancing the operational efficiency, accuracy, and data-driven decision-making within car wash systems.

Acknowledgments

First, I would like to thank the Almighty God for His countless blessings, directions and Mercy during this practicum project and report.

I have been blessed by my family with their unwavering support, encouragement and patience throughout all of this. I would like to express my gratitude to Prof. Dr. Utpal Kanti Das, Chair of the CSE Department, IUBAT – International University of Business Agriculture and Technology for his leadership and support during the practicum program. My special thanks go to my practicum supervisor Md. Khairul Islam for his valuable guidance, mentorship, constructive suggestions and encouragement throughout the development of this project and report.

I would like to express my gratitude to Shahinur Alam, Coordinator and Assistant Professor, CSE, IUBAT for his support and coordination during the practicum period. I would like to express my gratitude to AIQNIC for offering me an internship as well as having the chance to work on a real-time project related to Artificial Intelligence and Computer Vision. I would also like to express my gratitude to AIQNIC supervisors and team members for their knowledge sharing, technical guidance and development of my professional skills during the internship period.

I would also like to express my gratitude to my classmates, friends and colleagues of IUBAT for their cooperation, discussions and encouraging me throughout this journey.

Lastly, I would like to thank the international community of open-source developers and the creators of the tools and frameworks, including Python, PyTorch, OpenCV, PyQt5, Ultralytics YOLO, SQLite, and so many others that enable the creation of Smart Eye.

I would like to give my sincere gratitude and appreciation to all those who have helped, directly or indirectly, with this work.

Table of Contents

Letter of Transmittal	iii
Organization's Certificate.....	iv
Student's Declaration	v
Supervisor's Certification	vi
Abstract.....	vii
Acknowledgments	viii
List of Figures.....	xiii
List of Tables	xv
Chapter 1 Introduction.....	16
1.1 Background of the Study	17
1.2 Methodology	18
1.2.1 Primary Source.....	18
1.2.2 Secondary Sources.	19
1.3 Objectives	20
1.3.1 Broad Objective.	20
1.3.2 Specific Objectives.	20
1.4 Features	21
1.4.1 Monitoring and Detection Features.	21
1.4.2 Session Management Features.....	21
1.4.3 Alert and Notification Features.....	22
1.4.4 Analytics and Reporting Features.....	22
1.4.5 Configuration and Management Features.....	22
1.5 Proposed System Benefits.....	23
1.6 Process Model.....	24

1.6.1 Reasons for Choosing the Incremental Process Model.....	26
1.7 Feasibility Study	27
1.7.1 Technical Feasibility.....	27
1.7.2 Economic Feasibility.	27
1.7.3 Operational Feasibility.....	28
Chapter 2 Organizational Overview	29
2.1 Organizational Overview	30
2.2 Mission.....	30
2.3 Vision.....	31
2.4 Organization Services	31
2.5 My Position in this Organization	32
2.6 Address of the Organization	33
Chapter 3: Requirements Analysis.....	34
3.1 Requirement Engineering	35
3.2 Actor : Operator	35
3.3 Non-Functional Requirements	42
3.4 Hardware Requirements.....	43
3.5 Software Requirements	44
3.6 Use Case Diagram.....	44
Chapter 4: System Planning	47
4.1 System Project Planning	48
4.2 System Project Estimation	48
4.3 Identifying Complexity	51
4.4 Function Point Estimation.....	55

4.5 Calculation	57
4.6 Effort Distribution.....	61
4.7 Time Schedule	62
4.8 Cost Estimation.....	63
Chapter 5: Risk Management.....	67
5.1 Risk Management	68
5.2 Categories of Risk.....	69
5.3 The RMMM Plan.....	70
Chapter 6: Analysis & Modeling	79
6.1 System Architecture Design	80
6.2 Software Analysis Pattern.....	81
6.3 Activity Diagram	82
6.3.1 Activity Diagram: System Startup	84
6.3.2 Activity Diagram: Video Processing	85
6.3.3 Activity Diagram: Car Entry and Exit Detection.....	86
6.3.4 Activity Diagram: Alert Processing (Overtime)	87
6.3.5 Activity Diagram: Alert Processing (Vacuum Full)	88
6.3.6 Activity Diagram: Session History Viewing	89
6.3.7 Activity Diagram: Area Configuration	90
6.3.8 Activity Diagram: Model Loading and Switching.....	90
6.3.9 Activity Diagram: Settings Configuration	92
6.3.10 Activity Diagram: Analytics Dashboard.....	93
6.4 Swim Lane Diagram	94
6.5 Data Flow Diagram.....	96

6.5.1 Context Level DFD (Level 0)	96
6.5.2 Level 1 DFD	97
6.5.3 Level 2 DFD, Process 1: Video Capture.....	99
6.5.4 Level 2 DFD, Process 2: Object Detection.....	100
6.5.5 Level 2 DFD, Process 3: Vehicle Tracking	101
6.5.6 Level 2 DFD, Process 4: Area Monitoring	102
6.5.7 Level 2 DFD, Process 5: Session Logging	103
6.5.8 Level 2 DFD, Process 6: Alert Management	104
6.5.9 Level 2 DFD, Process 7: History and Analytics	105
6.5.10 Level 2 DFD, Process 8: Configuration Management.....	105
6.6 Entity Relationship Diagram.....	107
Chapter 7: Designing	109
Chapter 8: System Quality Management	113
8.1 System Quality Management.....	114
Chapter 9: Conclusion	118
9.1 Practicum and Its Value	119
9.2 Conclusion	120
9.3 Future Plan	121
References	123

List of Figures

Figure 3.1: Use Case Diagram for Smart Eye.....	46
Figure 4.1: Effort Distribution for Smart Eye.....	62
Figure 4.2: Time Schedule for Smart Eye	63
Figure 6.1: Activity Diagram: System Startup	84
Figure 6.2: Activity Diagram: Video Processing Loop	85
Figure 6.3: Activity Diagram: Car Entry and Exit Detection	86
Figure 6.4: Activity Diagram: Alert Processing (Overtime)	87
Figure 6.5: Activity Diagram: Alert Processing (Vacuum Full).....	88
Figure 6.6: Activity Diagram: Session History Viewing.....	89
Figure 6.7: Activity Diagram: Area Configuration.....	90
Figure 6.8: Activity Diagram: Model Loading and Switching.....	91
Figure 6.9: Activity Diagram: Settings Configuration	92
Figure 6.10: Activity Diagram: Analytics Dashboard	93
Figure 6.11: Swim Lane Diagram for Smart Eye	95
Figure 6.12: Context Level DFD (Level 0) for Smart Eye	97
Figure 6.13: Level 1 DFD for Smart Eye	98
Figure 6.14: Level 2 DFD, Process 1: Video Capture	99
Figure 6.15: Level 2 DFD, Process 2: Object Detection	100
Figure 6.16: Level 2 DFD, Process 3: Vehicle Tracking.....	101
Figure 6.17: Level 2 DFD, Process 4: Area Monitoring.....	102
Figure 6.18: Level 2 DFD, Process 5: Session Logging.....	103
Figure 6.19: Level 2 DFD, Process 6: Alert Management	104
Figure 6.20: Level 2 DFD, Process 7: History and Analytics	105
Figure 6.21: Level 2 DFD, Process 8: Configuration Management	106
Figure 6.22: Entity Relationship Diagram for Smart Eye.....	108
Figure 7.1: Detection Tab, Idle State	110
Figure 7.2: Detection Tab, Active State.....	110
Figure 7.3: Session History Dashboard	111

Figure 7.4: Analytics Dashboard	111
Figure 7.5: Settings and Configuration	112

List of Tables

Table 4.1: Complexity Matrix 1: External Inputs (EI)	49
Table 4.2: Complexity Matrix 2: External Outputs and External Queries (EO / EQ)	49
Table 4.3: Complexity Matrix 3: Transactional Function Weights	50
Table 4.4: Complexity Matrix 4: Internal Logical Files (ILF)	50
Table 4.5: Complexity Matrix 5: Data Function Weights (ILF / EIF).....	50
Table 4.6: Identifying Complexity (Transactional Function)	52
Table 4.7: Identifying Complexity (Data Function)	54
Table 4.8: Function, Input and Output (Operator)	55
Table 4.9: Unadjusted Function Point Contribution (Transactional Function)	58
Table 4.10: Unadjusted Function Point Contribution (Data Function).....	58
Table 4.11: Performance and Environmental Impact	59
Table 4.12: Function Point Calculation Summary.....	61
Table 4.13: Personal Cost	63
Table 4.14: Depreciation Calculation	64
Table 4.15: Software Cost.....	64
Table 4.16: Other Cost.....	66
Table 4.17: Account Table (Total Project Cost).....	66
Table 5.1: Project Risk 1	71
Table 5.2: Project Risk 2.....	72
Table 5.3: Technical Risk 1	73
Table 5.4: Technical Risk 2	74
Table 5.5: Technical Risk 3	75
Table 5.6: Technical Risk 4	76
Table 5.7: Business Risk 1	77
Table 5.8: Business Risk 2.....	78
Table 6.1: System Architecture Layers of Smart Eye.....	80

Chapter 1 Introduction

The car wash industry, like many service-oriented sectors, faces persistent operational challenges in tracking vehicle throughput, monitoring service durations, and managing area capacity in real time. In most conventional car wash facilities, vehicle counting and session management are performed manually, a process prone to human error, inconsistency, and loss of critical business intelligence. With the rapid advancement of computer vision and deep learning technologies (Redmon et al., 2016; Rivertz, 2021), it has become increasingly feasible to automate such monitoring tasks with high accuracy and low infrastructure cost.

This practicum project presents Smart Eye, an AI-powered desktop application developed to automate the monitoring of car wash operations. The system is built around a custom-trained YOLO11 object detection model (Redmon et al., 2016; Jocher, Chaurasia and Qiu, 2024), trained exclusively on annotated camera footage provided by the client, enabling highly accurate single-class vehicle detection tailored to the specific environment of the deployment site. Combined with the ByteTrack multi-object tracking algorithm (Zhang et al., 2022) and a custom Re-Identification (Re-ID) module (Ye et al., 2022), Smart Eye detects, tracks, and logs every vehicle that enters and exits the designated wash area. Built with a PyQt5 graphical user interface and backed by an SQLite database (Riverbed Technology, no date), the system provides operators with real-time session tracking, automated alerting, historical analytics, and configurable area management, all from a single desktop application.

1.1 Background of the Study

Car wash businesses operating in Bangladesh and similar developing markets typically rely on manual methods for recording vehicle entries, tracking how long each vehicle spends in the wash zone, and managing available parking or vacuum slots. These methods are inefficient and introduce significant gaps in operational data: vehicles may remain in the wash area beyond their allocated service time without any notification, slot occupancy is estimated rather than measured, and key performance metrics such as peak hours and average service duration are unavailable without time-consuming manual record-keeping.

The growing accessibility of high-performance object detection models, particularly the YOLO (You Only Look Once) family (Redmon et al., 2016), combined with robust

multi-object tracking algorithms such as ByteTrack (Zhang et al., 2022), has opened the door to affordable and effective video analytics solutions for small and medium-sized businesses. Unlike enterprise-grade surveillance systems that require dedicated servers and costly licenses, a Python-based desktop application can deliver real-time vehicle monitoring from existing CCTV or IP camera infrastructure on consumer-grade hardware.

A critical aspect of this project was the development of a domain-specific detection model. Rather than relying on a generic pretrained model, the client provided camera footage from their actual car wash facility. This footage was manually annotated to label each vehicle as the single target class 'Car', and used to fine-tune six YOLO model variants through transfer learning. All six models achieved a mAP@50 of approximately 99.49% on the validation set, confirming the high quality and consistency of the annotated dataset. Following a comprehensive comparison of detection accuracy, inference speed, and model size, YOLO11n was selected as the primary model for real-time deployment, with YOLO11l available as a higher-accuracy alternative for installations with more capable hardware.

1.2 Methodology

The development of Smart Eye and the preparation of this report were guided by one primary data source and a range of secondary sources drawn from existing literature and technical documentation.

1.2.1 Primary Source.

The sole primary data source for this project was the camera footage provided directly by the client, the car wash facility operator. This footage, captured from the facility's existing CCTV cameras, formed the foundation of both the model training pipeline and the system validation process. The following steps were carried out using this footage:

1. Frame extraction and manual annotation: Frames were extracted from the client-provided footage and annotated using Roboflow, labeling each vehicle with the single target class 'Car'. The annotated dataset was exported in YOLO11 format with train, validation, and test splits.

2. Custom model training: Six YOLO model variants, YOLOv9c, YOLOv10n, YOLO11n, YOLO11s, YOLO11m, and YOLO11l, were each trained for 20 epochs on the annotated dataset using transfer learning from pretrained weights. Training was performed on a GPU with an image size of 640×640 and a batch size of 2.
3. Model evaluation and selection: All six trained models were evaluated on the validation set and compared across mAP@50, mAP@50:95, precision, recall, inference time, model size, and parameter count. YOLO11n was selected as the primary deployment model based on its optimal balance of speed (72.1 FPS), size (5.2 MB), and accuracy (mAP@50 = 99.49%).
4. System validation: The complete Smart Eye application was validated against the same client footage to verify end-to-end detection accuracy, tracking stability, entry/exit detection reliability, and alert correctness under real car wash conditions.

1.2.2 Secondary Sources.

Secondary data was drawn from published research, official technical documentation, and reputable online resources to guide system design and implementation decisions:

1. Academic papers on the YOLO object detection architecture (Redmon et al., 2016; Wang et al., 2024) and the YOLO11 model documentation published by Ultralytics (Jocher, Chaurasia and Qiu, 2024).
2. The ByteTrack multi-object tracking paper (Zhang et al., 2022) for understanding the BYTE data association algorithm used for short-term tracking.
3. Official PyQt5 and Qt5 documentation for GUI design, signal-slot communication, and multi-threaded application development with QThread.
4. OpenCV documentation (Bradski and Kaehler, 2008) for video capture, frame processing, and image writing operations.
5. SQLite official documentation (Riverbed Technology, no date) for WAL-mode database design, schema definition, and concurrent query handling.
6. Roboflow documentation for dataset annotation workflow, augmentation pipeline, and YOLO-format dataset export.

1.3 Objectives

The objectives of Smart Eye are organized into a broad objective defining the overarching goal of the project, and specific objectives describing the measurable outcomes the system aims to achieve.

1.3.1 Broad Objective.

The broad objective of this practicum project is to design and develop an AI-powered desktop application that automates the real-time monitoring, tracking, and session logging of vehicles in a car wash facility, using a custom-trained detection model built from client-provided footage, thereby replacing manual record-keeping with an accurate, reliable, and operator-friendly computerized solution.

1.3.2 Specific Objectives.

The specific objectives of the project are as follows:

1. To collect, annotate, and prepare a domain-specific vehicle detection dataset from the client's camera footage using Roboflow, and to train and evaluate multiple YOLO model variants to select the best-performing model for the deployment environment.
2. To implement real-time vehicle detection using the custom-trained YOLO11 model, capable of accurately identifying cars in live video streams from both file-based sources and RTSP IP cameras.
3. To develop a robust multi-vehicle tracking system combining ByteTrack short-term tracking with a custom Re-ID module that maintains stable vehicle identities even after temporary occlusion or tracking loss.
4. To build an automated entry and exit detection system that records the timestamp, duration, and photographic snapshot of every vehicle wash session into an SQLite database.
5. To implement a configurable alert system that notifies operators when a vehicle exceeds the overtime threshold in the wash zone, or when the vacuum area reaches its defined capacity limit.

6. To provide a historical analytics dashboard presenting daily wash counts, average service durations, and hourly peak distribution, enabling data-driven operational decisions.

1.4 Features

Smart Eye provides a comprehensive set of features organized across five functional categories, each designed to address a specific operational need of car wash facility management.

1.4.1 Monitoring and Detection Features.

1. Live Video Feed: Displays a real-time annotated video feed in the Detection tab, showing bounding boxes, tracking IDs, wash area overlays, and park slot occupancy indicators.
2. Custom-Trained YOLO11 Detection: Detects vehicles using a YOLO11 model trained exclusively on annotated footage from the client's car wash facility, achieving 99.49% mAP@50 on a single 'Car' class. Both YOLO11n (72.1 FPS, 5.2 MB) and YOLO11l (26.4 FPS, 48.8 MB) variants are supported, selectable at runtime based on available hardware.
3. Multi-Vehicle Tracking: Tracks multiple vehicles simultaneously using ByteTrack, with a custom Re-ID module that reassigns stable identities to returning vehicles using cosine similarity matching on HOG-based feature vectors.
4. IP Camera Support: Supports RTSP IP camera streams in addition to file-based video sources, with automatic reconnection logic that retries up to 10 times upon stream loss.

1.4.2 Session Management Features.

1. Automated Entry and Exit Detection: Automatically detects when a washed vehicle enters or exits the defined park slot area using hysteresis-based point-in-polygon testing to eliminate false triggers.
2. Session Logging: Records every wash session into an SQLite database, capturing the tracker ID, entry time, exit time, total duration, session status, and file paths of entry and exit snapshots.
3. Snapshot Capture: Saves JPEG image snapshots of each vehicle at the moment of entry and exit detection for visual verification and record-keeping.

4. Session History Viewer: Provides a filterable table of all wash sessions with date range and status filters, session image previews, notes editing, and CSV export functionality.
5. Counter Management: Displays real-time counts of total entries, completed washes, and current vehicles in the vacuum area, with the ability to reset counters manually.

1.4.3 Alert and Notification Features.

1. Overtime Alert: Checks every 60 seconds for active sessions exceeding the configured overtime threshold and notifies the operator via an on-screen notification and a Windows system tray balloon message.
2. Vacuum Full Alert: Triggers an alert when the number of vehicles currently in the vacuum area reaches or exceeds the configured capacity threshold, preventing overcrowding.
3. Sound Alerts: Plays a system alert sound alongside visual notifications, with a configurable toggle to enable or disable audio alerts.

1.4.4 Analytics and Reporting Features.

1. Bar Chart, Cars Washed per Day: Renders a QPainter-based bar chart showing the daily count of completed wash sessions over the selected date range (7, 14, or 30 days).
2. Line Chart, Average Duration per Day: Displays a line chart of the average wash session duration per day, helping operators identify trends in service speed.
3. Heatmap, Peak Hours: Shows a 24-hour heatmap indicating the busiest hours of operation based on session entry times, enabling informed staffing decisions.
4. Summary Statistics Bar: Presents aggregated metrics including total sessions, busiest day, peak hour, and overall average duration at a glance.
5. CSV Export: Allows the operator to export the full session history or a filtered subset to a CSV file via a standard file save dialog.

1.4.5 Configuration and Management Features.

1. Interactive Area Configuration: Provides a dialog for drawing the wash zone and park slot boxes directly on a captured video frame using four-point polygon selection.

2. **Detection Settings:** Allows configuration of confidence threshold, IoU threshold, target FPS, and YOLO model file path from the Settings tab.
3. **Tracking Settings:** Exposes Re-ID similarity threshold, position threshold, and frame release timeout parameters for fine-tuning the tracking behavior.
4. **Model Switching:** Supports hot-swapping the YOLO model at runtime, for example, switching between YOLO11n and YOLO11l, without restarting the application.
5. **JSON Configuration Persistence:** Saves all settings to a JSON file with recursive default merging, value clamping validation, and corrupt-file recovery using a .bak rename strategy.

1.5 Proposed System Benefits

Smart Eye delivers measurable operational, financial, and managerial benefits to car wash facility operators. The proposed system benefits are outlined below:

Operational Efficiency:

1. Eliminates the need for a dedicated manual vehicle counter, reducing labor costs and human error in session recording.
2. Automates entry and exit detection with millisecond-precision timestamps, ensuring accurate service duration data for every vehicle.
3. Supports continuous monitoring from RTSP IP cameras with automatic reconnection, maintaining uninterrupted coverage without operator intervention.

Domain-Specific Accuracy:

5. Unlike generic pretrained models, the custom-trained YOLO11 detector was built exclusively from client-provided footage of the actual deployment environment, resulting in a 99.49% mAP@50, a level of accuracy well-suited for production use.
6. Six model variants were trained and benchmarked, giving operators the flexibility to choose between the lightweight YOLO11n (72.1 FPS, 5.2 MB) for speed and the YOLO11l (26.4 FPS, 48.8 MB) for maximum accuracy.

Real-Time Visibility and Control:

1. Provides operators with a live annotated video feed showing vehicle positions, tracking IDs, and area occupancy at a glance.
2. Delivers immediate overtime and vacuum full alerts through both on-screen notifications and system tray messages, enabling rapid response to capacity issues.
3. Displays up-to-date session counters, entries, completed washes, and vehicles in the vacuum zone, refreshed every 30 frames.
4. Data-Driven Business Intelligence:
5. Generates daily, weekly, and monthly analytics on wash throughput, average service duration, and peak operating hours, enabling evidence-based staffing and pricing decisions.
6. Maintains a permanent photographic record of every session's entry and exit snapshots, providing visual proof for dispute resolution or quality auditing.
7. Supports CSV export of session history for integration with external accounting or reporting tools.

Ease of Use:

1. The PyQt5 multi-tab interface is designed for non-technical operators, requiring no database knowledge or programming skills to use any feature of the system.
2. Configuration of detection areas is performed visually by clicking directly on a live video frame, with no need to enter coordinate values manually.
3. The Settings tab provides guided parameter configuration with automatic validation and a one-click reset-to-defaults option.

1.6 Process Model

Smart Eye was developed following the Incremental Process Model (Pressman and Maxim, 2014; Sommerville, 2016), a software engineering lifecycle model in which the system is designed, implemented, and tested in a series of small, self-contained increments.

Each increment delivers a working subset of the final system's functionality, which is then extended by the next increment until the complete system is realized.

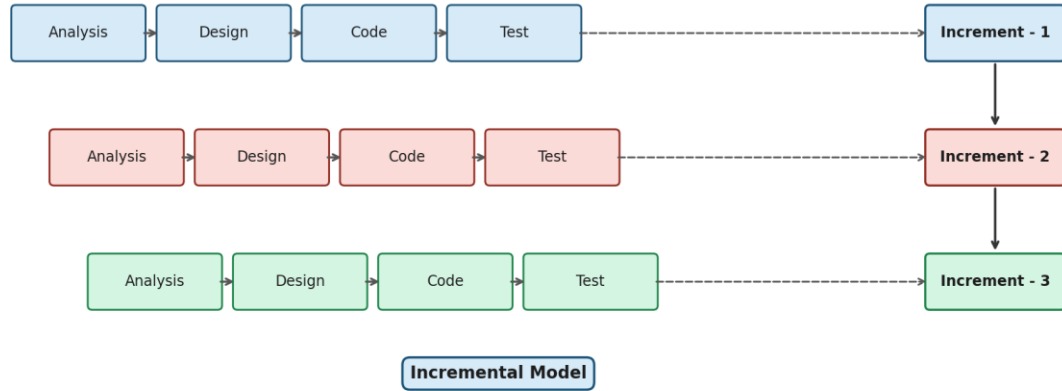


Figure 1.1: Incremental Process Model

The development of Smart Eye was organized into eight increments as follows:

1. Increment 1, Video Capture and PyQt5 GUI Framework: Established the core application skeleton including the multi-tab PyQt5 main window, the VideoProcessingThread for background video reading, and basic video source configuration.
2. Increment 2, Dataset Preparation and Custom Model Training: Extracted and annotated frames from the client-provided camera footage using Roboflow (single class: 'Car'). Trained six YOLO model variants, YOLOv9c, YOLOv10n, YOLO11n, YOLO11s, YOLO11m, and YOLO11l, for 20 epochs each using transfer learning on a GPU. Evaluated all models and selected YOLO11n as the primary deployment model based on its mAP@50 of 99.49%, inference speed of 72.1 FPS, and compact size of 5.2 MB.
3. Increment 3, YOLO11 Object Detection Integration: Integrated the custom-trained YOLO11 model via the Ultralytics library with configurable confidence and IoU threshold filtering, and added real-time annotated frame display in the Detection tab.
4. Increment 4, ByteTrack and Custom Re-ID Tracking: Added ByteTrack multi-object tracking and developed the custom EnhancedFuelStationTracker with HOG-based Re-ID feature extraction and cosine similarity matching for stable long-term vehicle identification.

5. Increment 5, Area Management and Entry/Exit Logic: Implemented the interactive area configuration dialog and the AreaTracker with hysteresis-based entry/exit detection for the wash zone and park slots.
6. Increment 6, SQLite Session Logging and Snapshots: Built the WAL-mode SQLite database layer, the EventLogger for session recording, and the snapshot saving mechanism using OpenCV.
7. Increment 7, Alert System: Implemented the overtime alert (QTimer, every 60 seconds) and vacuum full alert (stats_ready signal, every 30 frames) with system tray notifications and sound alerts.
8. Increment 8, Analytics, History Viewer, Configuration, and UI Polish: Developed the HistoryTab with filtering, image preview, notes editing, and CSV export; the AnalyticsTab with QPainter-rendered bar, line, and heatmap charts; the SettingsTab with JSON configuration persistence, validation, and default merging.

1.6.1 Reasons for Choosing the Incremental Process Model.

The Incremental Process Model (Pressman and Maxim, 2014; Sommerville, 2016) was selected for the development of Smart Eye over alternative models such as the Waterfall or Spiral models for the following reasons:

1. The system's functionality could be naturally decomposed into independent, deliverable increments. The dataset preparation and model training (Increment 2) could be completed and validated as a standalone milestone before the detection pipeline (Increment 3) depended on its output.
2. The core video processing pipeline, detection, tracking, and area monitoring, needed to be calibrated on real client footage before the dependent features of alerting and analytics could be built reliably on top of it. The incremental model allowed these foundational components to be stabilized early in the development cycle.
3. Unlike the Waterfall model, the incremental approach does not require all requirements to be finalized before implementation begins. Specific parameters such as Re-ID thresholds, hysteresis frame counts, and alert timing were determined empirically through iterative testing on the client's footage rather than through upfront specification.

4. Each completed increment produced a testable, working version of the system that could be demonstrated to the client for feedback, resulting in a product more closely aligned with actual operator needs.
5. The incremental model minimizes project risk by ensuring that a functional version of the core system remains available at all times. If a later increment encountered a technical obstacle, the previously completed increments remained intact and deliverable.

1.7 Feasibility Study

The feasibility of Smart Eye was assessed across three dimensions: technical, economic, and operational. The system was determined to be feasible in all three areas as discussed below.

1.7.1 Technical Feasibility.

Smart Eye is built entirely on mature, well-documented, and actively maintained open-source technologies. Python 3.10+ provides a rich ecosystem of libraries for computer vision, GUI development, and data management. The custom-trained YOLO11 model, developed using the Ultralytics framework, achieved a mAP@50 of 99.49% on the client's own camera footage, confirming that the detection task is technically solvable with the available data. The model supports both CPU and CUDA GPU inference, making it adaptable to a wide range of hardware configurations. OpenCV (Bradski and Kaehler, 2008) handles all video capture and frame processing operations. PyQt5 provides a robust, cross-platform GUI framework with native Windows integration for system tray notifications and sound alerts. SQLite, operating in WAL mode (Riverbed Technology, no date), delivers reliable concurrent read/write performance without requiring a dedicated database server. The system can be packaged into a standalone Windows executable using PyInstaller, eliminating the need for end-users to manage Python dependencies.

1.7.2 Economic Feasibility.

The development of Smart Eye incurs minimal cost. All software components, Python, PyQt5, OpenCV, the Ultralytics YOLO11 framework, Roboflow (free tier for annotation), and SQLite, are freely available at zero licensing cost. The model training was

performed on a single GPU using the client's own footage, eliminating the need to purchase any third-party dataset. The hardware requirements for deployment are modest: a standard desktop or laptop with at least 8 GB of RAM is sufficient, as the YOLO11n model runs at 72.1 FPS on a GPU and remains usable on CPU. The existing CCTV infrastructure of the car wash facility is leveraged directly via RTSP streams, requiring no additional camera investment. From an operational economics perspective, the system reduces the need for dedicated manual counters, prevents revenue leakage from untracked sessions, and generates actionable analytics that support more efficient resource allocation.

1.7.3 Operational Feasibility.

Smart Eye was designed with non-technical car wash operators as the primary users. The PyQt5 interface presents all monitoring information through a clear multi-tab layout: the Detection tab provides a live annotated video feed with real-time counters and alert notifications; the History tab allows operators to browse, filter, and export session records without any database knowledge; the Analytics tab presents visual charts automatically on a configurable auto-refresh timer; and the Settings tab provides guided configuration of all system parameters with input validation and a one-click reset-to-defaults option. Alert notifications are delivered through both on-screen overlays and Windows system tray balloon messages, ensuring operators are informed even when the application is minimized. Once monitoring is started, the system requires no continuous operator interaction, making it fully compatible with the typical workflow of a car wash attendant.

Chapter 2 Organizational Overview

2.1 Organizational Overview

AIQNIC is an emerging artificial intelligence and technology services company based in Savar, Dhaka, Bangladesh. Operating under the tagline "Engineering the Future of Intelligence," AIQNIC delivers end-to-end AI and data science solutions that enable organizations to harness the transformative power of machine learning, computer vision, and intelligent automation. The company was established with the purpose of bridging the gap between cutting-edge AI research and real-world business applications, making advanced technology accessible to organizations across various industries.

AIQNIC brings together a team of dedicated engineers, data scientists, and AI specialists who collaborate to develop bespoke solutions tailored to the specific operational needs of each client. The company's approach is grounded in a thorough understanding of both the technical landscape and the practical requirements of its clients, ensuring that every solution delivered is not only technically sound but also operationally relevant and measurable in its impact.

The company's portfolio spans a wide range of disciplines within artificial intelligence and data science, including predictive analytics, natural language processing, computer vision, and MLOps. AIQNIC's commitment to quality and continuous innovation positions it as a growing contributor to the technology sector in Bangladesh, serving clients who seek to modernize and automate their operational processes through intelligent software systems.

2.2 Mission

AIQNIC's mission is to engineer intelligent solutions that empower businesses to operate more efficiently, make data-driven decisions, and unlock new opportunities through the practical application of artificial intelligence and machine learning. The company is committed to delivering high-quality, production-ready AI systems that generate measurable value for clients across diverse industries. AIQNIC strives to make advanced AI capabilities accessible and affordable, ensuring that organizations of all sizes can benefit from the power of intelligent technology.

2.3 Vision

AIQNIC's vision is to become a leading artificial intelligence solutions provider in Bangladesh and the broader South Asian technology landscape. The company aspires to be recognized for its technical excellence, innovation-driven culture, and its ability to transform how businesses operate by making advanced AI capabilities practical and deployable. AIQNIC envisions a future where every business, regardless of scale, has access to intelligent systems that improve decision-making, increase productivity, and drive sustainable growth.

2.4 Organization Services

AIQNIC provides a comprehensive range of artificial intelligence and data science services. The key services offered by the organization are described below.

AI and Machine Learning Solutions: AIQNIC designs and develops custom AI and machine learning models tailored to specific business problems. From predictive analytics pipelines to recommendation engines and classification systems, the company provides end-to-end solutions covering data collection, model development, evaluation, and production deployment.

AI Automation: AIQNIC delivers intelligent automation solutions that enable businesses to streamline repetitive and time-consuming processes. By integrating AI-driven workflows into existing operational systems, the company helps clients reduce manual effort, minimize errors, and improve overall process efficiency.

Analytical and Data Science Solutions: The company offers comprehensive data science consulting and development services, including exploratory data analysis, statistical modelling, business intelligence dashboards, and advanced analytics. These solutions enable clients to derive actionable insights from their data and support evidence-based decision-making across business functions.

Natural Language Processing (NLP): AIQNIC develops NLP-based solutions including text classification, sentiment analysis, named entity recognition, and conversational AI systems. These solutions enable businesses to process and understand unstructured text data at scale, improving customer interaction quality and information management.

Computer Vision: AIQNIC specializes in building computer vision systems for object detection, image classification, and video analytics. These solutions are deployed across a range of real-world applications including automated monitoring, surveillance, and quality inspection. The Smart Eye vehicle monitoring system, developed during this practicum, is a direct application of the company's computer vision capabilities.

MLOps: AIQNIC provides MLOps services that bridge the gap between model development and production deployment. These services include model versioning, continuous integration and delivery pipelines for machine learning systems, production monitoring, and scalable infrastructure management for AI workloads.

2.5 My Position in this Organization

During my practicum at AIQNIC as a Junior AI Engineer, I was assigned to the AI and Machine Learning team under the supervision of the Technical Director. My primary responsibility was the design, development, and implementation of Smart Eye, a real-time computer vision-based vehicle monitoring system for car wash facilities.

Throughout the practicum period, my key responsibilities included:

- Designing and implementing the vehicle detection and tracking pipeline using YOLO11 and the ByteTrack multi-object tracking algorithm.
- Developing the custom appearance-based Re-Identification module using HOG feature descriptors and cosine similarity matching.
- Building the PyQt5 desktop application with four operational tabs covering real-time detection, session history, analytics, and system configuration.
- Designing and implementing the SQLite session database with Write-Ahead Logging for reliable, concurrent data persistence.
- Training and evaluating the custom YOLO11 detection model on a client-provided dataset of car wash facility footage.
- Conducting system testing, debugging, and performance optimization to meet the real-time processing requirements of the deployment environment.

Working at AIQNIC as a Junior AI Engineer provided an invaluable opportunity to apply theoretical knowledge from undergraduate studies to a real-world production system. The experience of collaborating within a professional AI team, adhering to engineering standards, and delivering a deployable system within a defined timeframe has significantly strengthened both technical competency and professional readiness in the field of artificial intelligence and software engineering.

2.6 Address of the Organization

House-177, Road-11, Block-B, Jahangirnagar Co-operative Housing Society, Savar, Dhaka-1343, Bangladesh.

Website: www.aiqnic.com

Chapter 3: Requirements Analysis

3.1 Requirement Engineering

Requirement engineering is a systematic process of identifying, documenting, and managing the requirements of a software system (Pressman and Maxim, 2014; Sommerville, 2016). It involves understanding the needs and expectations of stakeholders, analysing those needs, and translating them into clear and unambiguous requirements that guide the design, development, and implementation of the system. For Smart Eye, the requirements were gathered through direct consultation with the car wash facility owner and observation of the existing manual monitoring workflow.

This chapter documents all categories of requirements identified for Smart Eye. There are several types of requirements that are typically addressed in requirement engineering:

1. User Requirements
2. System Requirements
3. Functional Requirements
4. Non-Functional Requirements
5. Hardware Requirements
6. Software Requirements

3.2 Actor : Operator

Smart Eye is operated by a single actor, the car wash facility operator. The operator is the staff member responsible for monitoring vehicle activity, reviewing session history, and configuring the system. All functional capabilities of Smart Eye are designed to serve the needs of this actor.

User Requirement #1: The operator shall be able to start and stop the live video monitoring feed.

System Requirements:

- 1.1 The system shall accept a video source URL or device index configured in the Settings panel before monitoring begins.

- 1.2 The system shall initialise the YOLO11 detection model and the ByteTrack tracker when the operator starts monitoring.
- 1.3 The system shall continuously capture and display live video frames in the Detection tab while monitoring is active.
- 1.4 The system shall terminate all processing threads and release the video source when the operator stops monitoring.

Functional Requirements:

1. The Start button shall be disabled while the detection model is loading and re-enabled only when the model is ready.
2. Stopping monitoring shall finalise any open session records and return the interface to the idle state.
3. If the video source is unreachable at startup, the system shall display an error notification and remain in the idle state without crashing.

User Requirement #2: The operator shall be able to define the wash area and parking slot zones by drawing them interactively on the video frame.

System Requirements:

- 2.1 The system shall allow the operator to draw a wash-area rectangle on the video frame by clicking and dragging the mouse.
- 2.2 The system shall allow the operator to draw up to eight numbered parking-slot boxes on the video frame.
- 2.3 The system shall save all defined zone coordinates to the JSON configuration file immediately after drawing is completed.

The system shall reload and display all saved zone overlays on the video frame automatically on the next application startup.

Functional Requirements:

1. The operator shall be able to clear and redraw any zone without restarting the application.

2. Active zones shall be rendered as semi-transparent coloured overlays on the live video display.
3. A saved zone shall persist across application restarts until the operator explicitly redraws or clears it.
4. If no zones have been defined, the system shall display a prompt instructing the operator to draw zones before starting detection.

User Requirement #3: The operator shall be able to view real-time vehicle detection and tracking on the live video display.

System Requirements:

- 3.1 The system shall apply the YOLO11 object detection model to each processed frame and identify all vehicles present.
- 3.2 The system shall assign a persistent tracker ID to each detected vehicle using the ByteTrack multi-object tracking algorithm.
- 3.3 The system shall attempt to recover a vehicle's identity using appearance-based Re-Identification when tracking is temporarily lost due to occlusion or frame dropout.
- 3.4 The system shall overlay bounding boxes, tracker IDs, and zone occupancy indicators on the live video display in real time.

Functional Requirements:

1. Detection shall run at a configurable frame-skip interval to balance processing performance and detection accuracy.
2. Tracker IDs shall remain stable across consecutive frames for the same physical vehicle.
3. Re-Identification shall use HOG feature descriptors and cosine similarity against a stored appearance gallery.
4. A vehicle whose tracker ID is reassigned after re-acquisition shall retain its original session record and entry timestamp.

User Requirement #4: The operator shall be able to receive visual alerts when a vehicle exceeds the allowed wash time or when the vacuum bay is full.

System Requirements:

- 4.1 The system shall calculate the elapsed time for each vehicle inside the wash area on every processed frame.
- 4.2 The system shall raise a visual alert when a vehicle's wash duration exceeds the operator-configured overtime threshold.
- 4.3 The system shall monitor the vacuum bay occupancy counter and raise an alert when it reaches the configured maximum capacity.
- 4.4 The system shall sustain the overtime alert at a configurable repeat interval until the vehicle exits the wash area.

Functional Requirements:

1. Overtime alerts shall be displayed as a colour-coded banner in the Detection tab, identifying the affected vehicle by its tracker ID and elapsed time.
2. The alert banner shall be dismissed automatically when the vehicle exits the wash area.
3. Vacuum full alerts shall display the current number of vehicles occupying the vacuum bay alongside the configured maximum.
4. Alert thresholds shall be adjustable through the Settings tab and take effect without restarting the application.

User Requirement #5: The operator shall be able to browse the complete session history of all vehicles that have passed through the wash area.

System Requirements:

- 5.1 The system shall store a session record for every vehicle that passes through the wash area, including entry time, exit time, duration, status, parking slot assignment, and entry and exit snapshot image paths.
- 5.2 The system shall display all session records in a tabular view within the History tab.

5.3 The system shall allow the operator to filter session records by date range, status, or session duration.

5.4 The system shall allow the operator to view the entry and exit snapshot images for any selected session record.

Functional Requirements:

1. Session records shall be sortable by any column header in the history table.
2. The history table shall update automatically when new session records are added during active monitoring.
3. Applied filters shall take effect immediately without requiring a page reload or application restart.
4. Sessions flagged with an overtime status shall be visually distinguished from completed sessions in the table.

User Requirement #6: The operator shall be able to export session records to a CSV file for offline reporting.

System Requirements:

- 6.1 The system shall allow the operator to export all currently displayed session records to a CSV file at an operator-selected file path.
- 6.2 The exported file shall include all visible columns: session ID, vehicle ID, entry time, exit time, duration, status, and parking slot.

Functional Requirements:

1. The export shall apply the currently active filter state so that only records matching the current history view are written to the file.
2. A file-save dialog shall be presented to the operator to confirm the export destination before writing begins.
3. The system shall display a confirmation notification when the export completes successfully.

4. If the target path is not writable, the system shall display an error message without crashing.

User Requirement #7: The operator shall be able to view analytical charts and heatmaps showing vehicle throughput, session durations, and peak activity periods.

System Requirements:

- 7.1 The system shall render bar charts showing the number of vehicles processed per day and per hour.
- 7.2 The system shall render a line chart showing the trend of average session durations over time.
- 7.3 The system shall render a heatmap showing peak vehicle activity across days of the week and hours of the day.
- 7.4 The system shall refresh all charts automatically on a periodic timer while the Analytics tab is the active view.

Functional Requirements:

1. All charts shall be generated from the session records stored in the SQLite database.
2. The Analytics tab shall display summary statistics including total sessions, overall average duration, and the busiest recorded hour.
3. All charts shall render gracefully when the session database is empty, displaying an appropriate zero-state message.
4. Chart data shall reflect all session records in the database unless a date filter is applied.

User Requirement #8: The operator shall be able to switch between different trained YOLO detection models at runtime without restarting the application.

System Requirements:

- 8.1 The system shall allow the operator to select a different YOLO model checkpoint file through a file-picker control in the Settings tab.
- 8.2 The system shall load the selected model in a background thread without interrupting the active video processing pipeline.

8.3 The system shall release the previously loaded model from memory before loading the replacement to prevent resource conflicts.

Functional Requirements:

1. The model file picker shall display only .pt model files from the configured model directory.
2. A loading progress indicator shall be displayed in the notification panel while the new model is being initialised.
3. If the selected model file is invalid or incompatible, the system shall retain the current model and display an error notification.
4. The newly loaded model shall take effect on the next processed frame after initialisation completes.

User Requirement #9: The operator shall be able to adjust detection parameters such as confidence threshold, IOU threshold, frame-skip rate, and overtime limit through the Settings panel.

System Requirements:

- 9.1 The system shall allow the operator to adjust the confidence threshold, IOU threshold, frame-skip rate, and overtime duration limit through the Settings tab.
- 9.2 The system shall save all confirmed configuration changes to the JSON settings file immediately.
- 9.3 The system shall apply updated parameter values on the next processed frame without restarting the application or the detection pipeline.

Functional Requirements:

1. Each configuration field shall display the current active value when the Settings tab is opened.
2. The confidence threshold and IOU threshold fields shall only accept numeric values between 0.0 and 1.0.

3. The overtime duration field shall only accept positive integer values representing seconds.
4. The system shall display a validation error message and reject invalid inputs without writing them to the configuration file.

User Requirement #10: The operator shall be able to view system status messages, including model loading progress, detection events, and tracking notifications.

System Requirements:

- 10.1 The system shall display timestamped status messages in the notification panel of the Detection tab for all significant system events.
- 10.2 The system shall colour-code messages by severity: green for informational events, amber for warnings, and red for errors.
- 10.3 The notification panel shall log events including model loading progress, vehicle entry and exit detections, Re-Identification match results, alert triggers, and database write confirmations.

Functional Requirements:

1. The notification panel shall be scrollable, allowing the operator to review all messages recorded during an active monitoring session.
2. New messages shall be appended to the bottom of the panel and the view shall scroll automatically to display the latest entry.
3. Error-level messages shall remain visible in the panel until the operator manually clears them or the application is restarted.

3.3 Non-Functional Requirements

Non-functional requirements are the constraints that specify the quality attributes of the Smart Eye system. They define how well the system performs its functions rather than what specific functions it performs.

Performance: The system shall process and display video frames at a minimum of 20 frames per second on the target hardware to ensure smooth real-time monitoring.

Reliability: The system shall recover from temporary tracking loss and re-identify vehicles within a configurable timeout window without operator intervention.

Accuracy: The vehicle detection model shall achieve a minimum mean Average Precision (mAP@50) of 95 percent on the validation dataset.

Usability: The operator interface shall allow a non-technical user to start monitoring, draw zones, and review session history with no prior training beyond a five-minute orientation.

Scalability: The system shall support monitoring of up to eight distinct parking slots simultaneously without degradation in detection or tracking performance.

Portability: The system shall run on any Windows 10 or Windows 11 machine equipped with Python 3.10 or later and an NVIDIA GPU, with all dependencies installable via pip.

Maintainability: The system codebase shall be organised into clearly separated modules for detection, tracking, area logic, event logging, and UI so that individual components can be replaced or updated independently.

Data Integrity: The SQLite database shall use Write-Ahead Logging (WAL) mode to prevent data corruption in the event of an unexpected application shutdown.

Security: The system shall store all session data locally on the operator's machine and shall not transmit any video feed or session records over a network.

Responsiveness: The graphical user interface shall remain interactive during video processing; all background tasks shall run in separate threads so that the UI does not freeze.

3.4 Hardware Requirements

The following minimum hardware specifications are required to run Smart Eye at acceptable real-time performance:

- Processor: Intel Core i5 (8th generation or later) or equivalent AMD Ryzen processor
- Memory: Minimum 8 GB RAM (16 GB recommended for concurrent multi-slot monitoring)
- GPU: NVIDIA GPU with CUDA support (GTX 1060 or better) for real-time YOLO inference; CPU-only mode is supported but performance will be reduced

- Storage: Minimum 10 GB of free disk space for the application, YOLO model files, session snapshots, and the SQLite database
- Camera: IP camera or USB webcam capable of delivering a minimum 720p resolution video stream
- Display: Monitor with a minimum resolution of 1280 by 720 pixels to accommodate the PyQt5 interface

3.5 Software Requirements

Smart Eye depends on the following software components:

- Operating System: Windows 10 or Windows 11 (64-bit)
- Python 3.10 or later: core runtime environment
- PyTorch 2.x with CUDA support: deep learning inference backend for YOLO11
- Ultralytics YOLO 11: model architecture, training, and inference framework
- OpenCV (cv2): video capture, frame processing, and image manipulation
- PyQt5: graphical user interface framework
- SQLite 3: embedded relational database for session storage (included with Python standard library)
- NumPy: numerical array operations for feature extraction and tracking calculations
- Matplotlib: chart rendering for the Analytics tab
- SciPy: cosine similarity computation for the Re-Identification module
- Roboflow: cloud annotation platform used during the dataset preparation phase (not required at runtime)

3.6 Use Case Diagram

A Use Case Diagram provides a high-level visual summary of the interactions between system actors and the functional capabilities of the system. The diagram uses three standard UML notations (Booch, Rumbaugh and Jacobson, 2005):

- Actor: represented by a stick figure, denoting an external entity (person or system) that interacts with the system
- Use Case: represented by an oval, denoting a specific action or service that the system provides to the actor
- System Boundary: represented by a rectangle enclosing all use cases, denoting the scope of the system being modelled

The Use Case Diagram for Smart Eye is shown below. The single actor is the Operator (the car wash facility staff member), who interacts with the system through the PyQt5 desktop application. All primary use cases correspond to the functional requirements identified in Section 3.3.

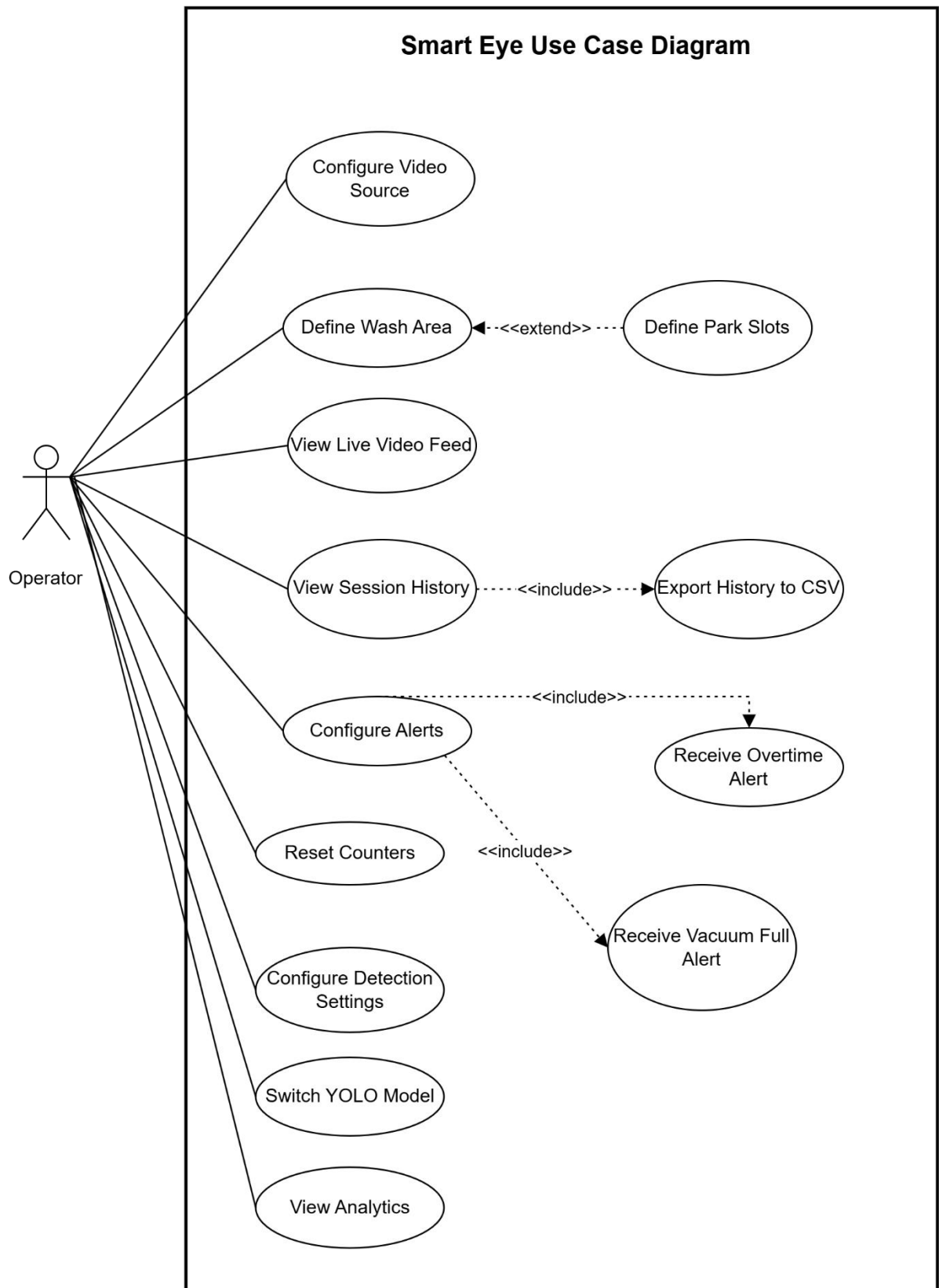


Figure 3.1: Use Case Diagram for Smart Eye

Chapter 4: System Planning

This chapter covers the project planning activities undertaken for Smart Eye, including function point estimation, complexity identification, effort distribution, time scheduling, and cost estimation. The project delivers a desktop monitoring application running on the Windows platform with GPU-accelerated real-time vehicle detection and tracking.

4.1 System Project Planning

Before starting any software project, it is crucial to engage in project planning activities. Software project planning involves several key activities that help estimate the work required, resources needed, and time duration, and analyse the feasibility of the project. These activities ensure that the project is well-understood, properly scoped, and has a realistic plan for execution.

The activities of this project's software project planning are as follows:

- System Project Estimation
- Function Point Estimation
- Based Estimation
- Effort Distribution
- Task Scheduling Project
- Cost Estimation

4.2 System Project Estimation

For projects to be planned and carried out successfully, software project estimating is essential. A software project estimate's accuracy is influenced by a number of variables:

Properly estimated size of the product: Estimating the size of the software product to be built is essential. This involves breaking down the project into smaller components and determining the effort required for each component. Accurate size estimation helps in resource allocation, scheduling, and budgeting.

Alignment with the software team's capabilities: The project plan should take into account the engineers' level of productivity, experience, and domain knowledge. For Smart Eye the developer required expertise in computer vision, deep learning, PyQt5 GUI

programming, and SQLite database management. The accuracy of the estimate is improved by matching the project schedule to these skill requirements.

Product requirements' stability: Requirements that change frequently lead to project delays and additional work. For Smart Eye, the core requirements, specifically real-time vehicle detection, wash-area tracking, session logging, and analytics, were established and frozen before implementation began, providing a stable estimation baseline.

The following standard complexity matrices are used to classify each function point according to its File Type References (FTR) and Data Element Types (DET):

Table 4.1: Complexity Matrix 1: External Inputs (EI)

DATA ELEMENTS	1–4	5–15	> 15
0–1	Low	Low	Average
2	Low	Average	High
3 or more	Average	High	High

Table 4.2: Complexity Matrix 2: External Outputs and External Queries (EO / EQ)

DATA ELEMENTS	1–5	6–19	> 19
0–1	Low	Low	Average
2–3	Low	Average	High
> 3	Average	High	High

Table 4.3: Complexity Matrix 3: Transactional Function Weights

FUNCTION TYPE	EO	EQ	EI
Low	4	3	3
Average	5	4	4
High	7	6	6

Table 4.4: Complexity Matrix 4: Internal Logical Files (ILF)

DATA ELEMENTS	1–19	20–50	> 50
1	Low	Low	Average
2–5	Low	Average	High
> 5	Average	High	High

Table 4.5: Complexity Matrix 5: Data Function Weights (ILF / EIF)

VALUES	ILF	EIF
Low	7	5
Average	10	7
High	15	10

4.3 Identifying Complexity

Identifying the complexity of the project is an important step in software project planning. One approach to assessing complexity is by counting function points (IFPUG, 2010). Function points are a measure of the functionality provided by a software application and help estimate the size and effort required for development.

In function point analysis there are two main categories of functions: Transactional Functions and Data Functions.

1. Transactional Functions:

- External Inputs (EI): User-initiated inputs that enter the system and update internal files. Examples in Smart Eye include configuring the wash area zone, adjusting detection thresholds, and switching the YOLO model.
- External Outputs (EO): The system's responses or outputs presented to the user. Examples include the live annotated video frame, the overtime alert banner, snapshot images, and analytics charts.
- External Queries (EQ): Inquiries or requests for information that retrieve data without modifying it. Examples include viewing the session history table and querying park slot occupancy.

2. Data Functions:

- Internal Logical Files (ILF): Internal data maintained by the system. Smart Eye maintains the wash_sessions SQLite table, the JSON configuration file, and the on-disk JPEG snapshot archive.
- External Interface Files (EIF): Files used by the system but maintained externally. Smart Eye references the pre-trained YOLO11 model checkpoint file and the live camera video stream.

Supporting metrics used to determine complexity ratings:

- File Type Referenced (FTR): The number of different internal or external file types referenced by a transaction or data function.

- Data Element Types (DET): The number of unique data fields referenced or processed by a transaction or data function.
- Record Element Types (RET): The number of unique record types within a data function.

Based on the complexity matrices above, each transactional function of Smart Eye is evaluated using FTR and DET counts to determine a Low, Average, or High complexity rating. Table 4.7 shows the result of this classification for all External Inputs (EI), External Outputs (EO), and External Queries (EQ).

Table 4.6: Identifying Complexity (Transactional Function)

No.	Function	Type	FTR	DET	Complexity
1	Start / Stop Video Processing	EI	1	2	Low
2	Adjust Confidence Threshold	EI	1	2	Low
3	Adjust Overtime Limit	EI	1	2	Low
4	Configure Frame Skip Rate	EI	1	2	Low
5	Draw Wash Area Zone	EI	2	5	Average

6	Switch YOLO Model	EI	2	3	Average
7	Update Settings	EI	2	8	Average
8	Overtime Alert	EO	1	3	Low
9	Log Entry Snapshot	EO	2	4	Low
10	Log Exit Snapshot	EO	2	4	Low
11	Display Status Notifications	EO	1	3	Low
12	Detect Vehicle	EO	2	8	Average
13	Track Vehicle	EO	2	6	Average
14	Export CSV	EO	2	8	Average
15	View Analytics Charts	EO	3	12	High

16	View Session History	EQ	2	11	Average
17	Detect Wash Zone Entry	EQ	2	8	Average
18	Detect Wash Zone Exit	EQ	2	8	Average
19	Detect Park Slot Occupancy	EQ	2	7	Average
20	Re-ID After Tracking Loss	EQ	3	10	High

Table 4.7: Identifying Complexity (Data Function)

Table 4.7 presents the complexity classification for the Data Functions, specifically Internal Logical Files (ILF) and External Interface Files (EIF), using RET and DET counts.

No.	Function	Type	RET	DET	Complexity
1	wash_sessions Table	ILF	1	11	Low
2	Config JSON File	ILF	1	14	Low
3	Snapshot	ILF	1	4	Low

	Files				
4	YOLO11 Model File	EIF	1	5	Low
5	Video Source Stream	EIF	1	3	Low

4.4 Function Point Estimation

The Function Point Estimation for Smart Eye is presented below. Since the system has a single user role, the Operator, and all functionalities are listed under one table.

Table 4.8: Function, Input and Output (Operator)

Functionalities	Input	Output
Start / Stop Video Processing	Start / Stop button press	Live video feed displayed on Detection tab
Draw Wash Area Zone	Mouse drag coordinates on video frame	Green rectangle and park slot boxes saved to config
Detect Vehicle	Processed video frame	Bounding boxes with tracker IDs overlaid on frame
Track Vehicle	Bounding box sequence across frames	Persistent tracker ID maintained per vehicle
Re-ID After Tracking Loss	Appearance feature vector +	Original tracker ID restored;

	position	session continuity preserved
Detect Wash Zone Entry	Vehicle centroid enters green zone	Session record created in wash_sessions table
Detect Wash Zone Exit	Vehicle centroid leaves green zone	Session record updated with exit time and duration
Detect Park Slot Occupancy	Vehicle centroid + slot box coordinates	Slot number assigned and stored in session record
Overtime Alert	Elapsed time in wash zone	Colour-coded alert banner displayed in Detection tab
Log Entry Snapshot	Entry event trigger	JPEG image captured and path saved to session record
Log Exit Snapshot	Exit event trigger	JPEG image captured and path saved to session record
View Session History	Click History tab	All wash_sessions records shown in sortable table
Export CSV	Export button + file path selection	CSV file written to chosen directory
View Analytics Charts	Click Analytics tab	Bar chart, line chart, and heatmap rendered from DB data

Switch YOLO Model	Model file path from Settings	New model loaded in background thread
Adjust Confidence Threshold	Slider / numeric input value	Detection sensitivity updated on next frame
Adjust Overtime Limit	Numeric input value in minutes	Alert trigger threshold updated in config
Configure Frame Skip Rate	Numeric input value	Processing rate updated; config saved
Update Settings	All settings panel inputs	JSON configuration file saved and applied
Display Status Notifications	System events (model load, DB writes)	Timestamped colour-coded messages shown in panel

4.5 Calculation

Using the complexity matrices defined in Section 4.2 and the complexity ratings identified in Section 4.3, each function of Smart Eye is assigned a weight. The Unadjusted Function Points (UFP) (IFPUG, 2010) are then calculated by multiplying the count of each complexity class by its standard weight and summing the results.

Table 4.9 shows the UFP contribution from the three transactional function types (EI, EO, EQ), broken down by complexity level.

Table 4.9: Unadjusted Function Point Contribution (Transactional Function)

Function Type	Low $\times$ Weight	Average Weight $\times$	High $\times$ Weight	Total FP
External Inputs (EI)	$4 \times 3 = 12$	$3 \times 4 = 12$	$0 \times 6 = 0$	24
External Outputs (EO)	$4 \times 4 = 16$	$3 \times 5 = 15$	$1 \times 7 = 7$	38
External Queries (EQ)	$0 \times 3 = 0$	$4 \times 4 = 16$	$1 \times 6 = 6$	22
UFP Subtotal (Transactional Functions)				84

Table 4.10: Unadjusted Function Point Contribution (Data Function)

Table 4.10 shows the UFP contribution from the two data function types (ILF and EIF), broken down by complexity level.

Function Type	Low $\times$ Weight	Average Weight $\times$	High $\times$ Weight	Total FP
Internal Logical Files (ILF)	$3 \times 7 = 21$	$0 \times 10 = 0$	$0 \times 15 = 0$	21
External Interface Files (EIF)	$2 \times 5 = 10$	$0 \times 7 = 0$	$0 \times 10 = 0$	10
UFP Subtotal (Data Functions)				31

The combined Unadjusted Function Points (UFP) for Smart Eye is: $UFP = 84$ (Transactional) + 31 (Data) = 115.

The Technical Complexity Factor (TCF) is calculated using 14 General System Characteristics (GSC), each rated on a scale of 0 (no influence) to 5 (essential influence). The sum of all 14 ratings (ΣFi) determines the Value Adjustment Factor (VAF). Table 4.11 presents the rating and justification for each characteristic.

Table 4.11: Performance and Environmental Impact

No.	General System Characteristic	Rating (0–5)	Justification
1	Data Communications	3	Moderate: single local camera stream
2	Distributed Data Processing	0	No distribution; all processing is local
3	Performance	5	Essential: real-time processing at ≥ 20 FPS
4	Heavily Used Configuration	3	GPU + memory constraints must be managed
5	Transaction Rate	4	High: 25+ detection cycles per second
6	Online Data Entry	3	Zone drawing and settings entry at runtime

7	End User Efficiency	4	Non-technical operator must use with ease
8	Online Update	4	DB updated live on each entry/exit event
9	Complex Processing	5	Essential: ML inference, Re-ID, tracking
10	Reusability	2	Modules are reusable but not primary goal
11	Installation Ease	3	pip install + config required
12	Operational Ease	4	GUI designed for non-technical operator
13	Multiple Sites	0	Single-site desktop deployment
14	Facilitate Change	3	Settings panel supports runtime parameter changes
Total ΣFi		43	

$$\text{Value Adjustment Factor (VAF)} = 0.65 + (0.01 \times \Sigma Fi) = 0.65 + (0.01 \times 43) = 1.08$$

$$\text{Adjusted Function Points (FP)} = \text{UFP} \times \text{VAF} = 115 \times 1.08 = 124.2$$

Lines of Code (LOC) Estimation using Python conversion rate (50 LOC/FP):

$$\text{LOC} = \text{FP} \times 50 = 124.2 \times 50 = 6210 \text{ lines (6.21 KLOC)}$$

COCOMO Basic Model (Pressman and Maxim, 2014) (Organic Mode):

$$\rightarrow \text{Effort} = 2.4 \times (\text{KLOC})^{1.05} = 2.4 \times (6.21)^{1.05} \approx 16.3 \text{ Person-Months}$$

$$\rightarrow \text{Duration} = 2.5 \times (\text{Effort})^{0.38} = 2.5 \times (16.3)^{0.38} \approx 7.2 \text{ Months}$$

$$\rightarrow \text{Average Staff} = \text{Effort} \div \text{Duration} = 16.3 \div 7.2 \approx 2.3 \text{ (1 developer)}$$

Function Point Calculation Summary:

Table 4.12: Function Point Calculation Summary

Metric	Value
Unadjusted Function Points (UFP)	115
ΣF_i (sum of 14 GSC ratings)	43
Value Adjustment Factor (VAF)	1.08
Adjusted Function Points (FP)	124.2
Estimated Effort (COCOMO)	16.3 Person-Months

4.6 Effort Distribution

The total estimated effort of approximately 16 person-months is distributed across the major development phases of Smart Eye as shown in Figure 4.1. Implementation (coding) represents the largest share given the complexity of the detection, tracking, and Re-ID modules. Model training and evaluation also carries a significant portion due to the iterative nature of testing six YOLO model variants.

Figure 4.1: Effort Distribution for Smart Eye

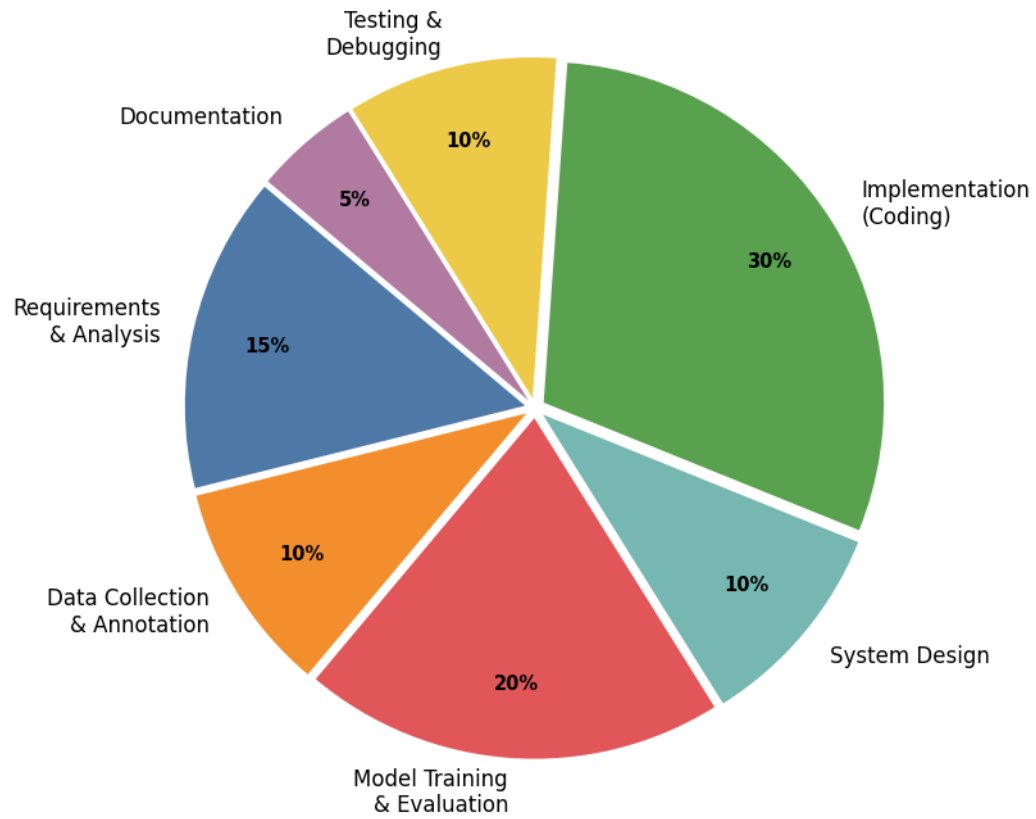


Figure 4.1: Effort Distribution for Smart Eye

4.7 Time Schedule

The project was executed over twelve weeks. Figure 4.2 shows the Gantt chart illustrating the start and duration of each major task. Several tasks overlap to make effective use of time; for example, GUI development began while the tracking module was still being refined.

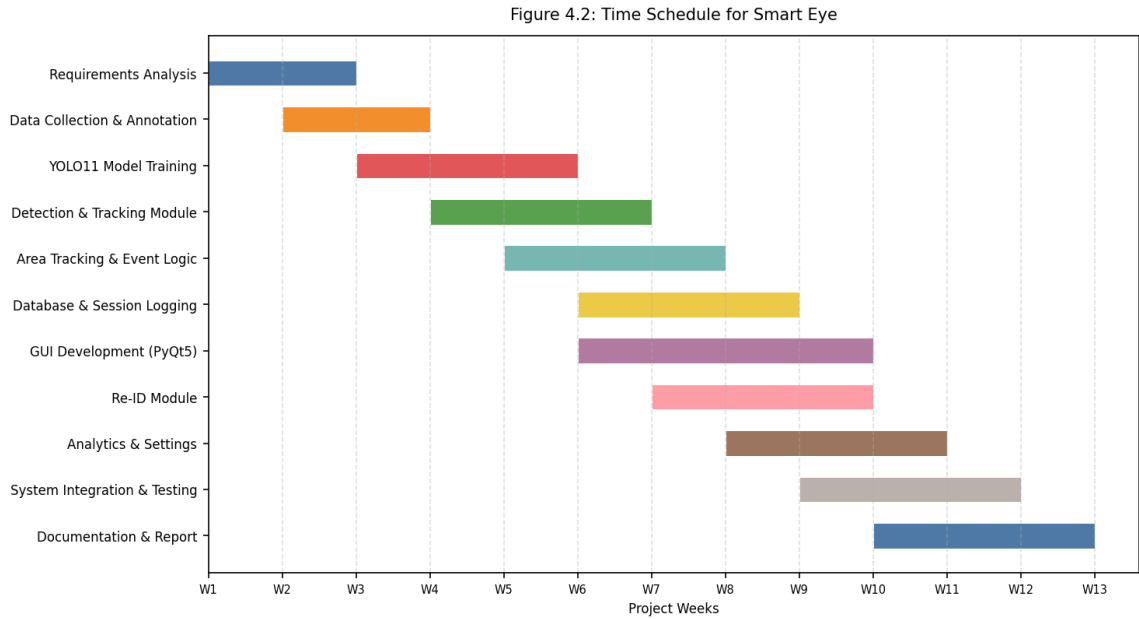


Figure 4.2: Time Schedule for Smart Eye

4.8 Cost Estimation

The total project cost covers the developer's time, hardware depreciation, software licences, and miscellaneous operational expenses. Since Smart Eye uses exclusively open-source tools, software licensing costs are zero.

Personal Cost:

Table 4.13: Personal Cost

Role	Head Count	Working Days	Hours/Day	Rate (BDT/hr)	Total (BDT)
Software Developer	1	90	8	100	72,000
Total Personal Cost					72,000 BDT

Depreciation Calculation:

Table 4.14: Depreciation Calculation

Item	Purchase Cost (BDT)	Useful Life (Years)	Monthly Dep. (BDT)	Months Used	Total Dep. (BDT)
Laptop + RTX GPU	1,20,000	5	2,000	3	6,000
Web Camera	3,500	3	97	3	291
External SSD (storage)	8,000	4	167	3	501
Total Hardware Depreciation					6,792 BDT

Software Cost:

Table 4.15: Software Cost

Software / Tool	Purpose	Licence	Cost (BDT)
Python 3.11	Core runtime	Open-source	0
PyTorch 2.x + CUDA	Deep learning inference	Open-source	0
Ultralytics YOLO 8.x	Model training and inference	AGPL-3.0	0

OpenCV (cv2)	Video capture and image processing	Open-source	0
PyQt5	GUI framework	GPL/LGPL	0
SQLite 3	Session database	Public Domain	0
NumPy / SciPy	Numerical and Re-ID operations	Open-source	0
Matplotlib	Analytics chart rendering	Open-source	0
Roboflow (free tier)	Dataset annotation and export	Free tier	0
VS Code	Development IDE	Free	0
Total Software Cost			0 BDT

Other Cost:

Table 4.16: Other Cost

Expense	Monthly Cost (BDT)	Duration (Months)	Total (BDT)
Internet Service	700	3	2,100
Electricity	500	3	1,500
Printing & Binding	N/A	N/A	500
Total Other Cost			4,100 BDT

Account Table (Total Project Cost Summary):

Table 4.17: Account Table (Total Project Cost)

Cost Category	Amount (BDT)
Personal Cost	72,000
Hardware Depreciation	6,792
Software Cost	0
Other Cost	4,100

Chapter 5: Risk Management

This chapter presents the risk management framework (Charette, 1989) applied to the Smart Eye project. Risk management is a systematic process of identifying, classifying, assessing, and mitigating potential threats that could affect the project's schedule, quality, or success. Given the real-time computer vision nature of Smart Eye, which depends on GPU hardware, trained deep learning models, live video feeds, and continuous database writes, careful risk planning is essential to ensure reliable system operation.

5.1 Risk Management

Risk management is a crucial aspect of any software project as it involves identifying and addressing potential risks that may arise (Pressman and Maxim, 2014; Charette, 1989; Sommerville, 2016). Failing to properly analyse and estimate project risks can lead to numerous problems including schedule delays, cost overruns, and system failures. Therefore, it is important to implement effective risk management strategies throughout the project lifecycle.

The process of risk management consists of several stages aimed at understanding and managing uncertainties. These stages help the development team in identifying, assessing, and mitigating risks before they materialise into actual problems.

The following phases are typically followed to establish a risk management model:

Identification: This entails gathering data in order to identify potential dangers. Information regarding potential risks and hazards can be gathered using a variety of automated and human methods. For Smart Eye, risks were identified by reviewing the technology stack, hardware dependencies, and operational environment of the car wash facility.

Classification: Risk classification requires the creation of a structured model to categorise risks and align observable risk characteristics with that model. It is possible to assess and categorise hazards using both quantitative and qualitative methods. Risks for Smart Eye are classified into three categories: Project Risks, Technical Risks, and Business Risks.

Assessment: Risk assessment focuses on identifying pertinent risk scenarios or events that may result in loss or damage. It entails calculating the likelihood that these occurrences will occur. A thorough risk analysis should provide appropriate recommendations and be transparent, logical, and consistent.

Analysis: The possible effects of risk patterns or scenarios are determined by risk analysis, including the size of prospective losses and the recovery costs involved. It analyses weaknesses and takes into account the project's risk tolerance, which results in the creation of mitigation measures.

Implementation: In order to manage and address identified risks, risk management implementation requires developing policies, processes, and methods. It should establish a balance between the expense of preventing damage or loss and the direct and indirect costs associated with maintaining the value of project assets. For Smart Eye, considerations include hardware stability, GPU driver compatibility, model performance under varying environmental conditions, and database integrity.

By following a comprehensive risk management approach, Smart Eye can effectively address potential risks and minimise their impact on the project's success.

5.2 Categories of Risk

It is crucial to take into account a variety of risk categories in any software project. For Smart Eye, the following three risk categories have been identified and taken into consideration:

1. **Project Risks:** These risks threaten the project plan. If they occur, the project's schedule will likely be delayed and costs could increase. Assessing project risks for Smart Eye entails detecting potential problems with the project timeline, available hardware resources, scope definition, and documentation quality.

2. **Technical Risks:** These risks put the quality and timeliness of the software under development in jeopardy. For Smart Eye, technical risks arise from the complexity of

combining real-time YOLO inference, multi-object tracking, Re-ID, area monitoring, and live database writes within a single desktop application. Issues with GPU availability, model accuracy, tracking continuity, and video source stability all fall into this category.

3. Business Risks: These risks threaten the viability and adoption of the system once deployed. Business risks for Smart Eye include operator resistance to adopting the technology, privacy concerns related to video surveillance in the workplace, and the risk of key third-party open-source libraries being deprecated or undergoing breaking changes in future versions.

By recognising and addressing risks in these categories, Smart Eye can proactively manage uncertainties and make informed decisions to mitigate potential negative impacts.

5.3 The RMMM Plan

The RMMM (Risk Mitigation, Monitoring, and Management) Plan (Pressman and Maxim, 2014; Sommerville, 2016) is an essential component of risk management in a software project. It outlines the strategies and actions used to address risks throughout the project lifecycle. The following three components are included in the RMMM Plan:

1. Risk Mitigation: This involves proactive planning to avoid or minimize the occurrence and impact of identified risks. Mitigation strategies may include preventive measures, contingency plans, alternative approaches, and risk reduction techniques. The goal is to reduce the probability and severity of risks before they occur.

2. Risk Monitoring: Continuous assessment of predicted risks to determine if they occur and to ensure that preventive measures are effectively implemented. Monitoring also involves collecting relevant information for future risk analysis and identifying the relationship between risks and their associated problems. By monitoring risks, the development team can take timely actions to address emerging issues.

3. Risk Management: This refers to the actions taken when mitigation measures have failed and a risk becomes an actual problem. Risk management involves implementing predefined response plans, activating contingency measures, and mobilizing resources to

address the risk and minimise its impact on the project. The focus is on resolving the issue and minimising any negative consequences.

Type of Impact: Impact assessment is a crucial aspect of risk management. The RMMM Plan categorises the impact of risks into different levels based on their severity. The impact levels are defined as follows:

- **Catastrophic (1):** Risks that have severe consequences, leading to significant disruptions or failures in project objectives.
- **Marginal (2):** Risks that have minimal impact on the project, causing minor setbacks or delays but can be managed within the project's existing capabilities.
- **Tolerable (3):** Risks that have a moderate impact on the project, resulting in noticeable challenges and requiring additional resources or adjustments to overcome.
- **Critical (4):** Risks that have a significant impact on project success and require immediate attention and decisive actions to prevent major failures.

Type of Probability: very low (<10%), low (10–25%), moderate (25–50%), high (50–75%), very high (>75%).

Project Risks: The following project risks have been identified and managed for Smart Eye.

Table 5.1: Project Risk 1

Name	Requirement Scope Creep
Probability	Low (10–25%)
Impact	Marginal
Description	The car wash operator may request additional monitoring zones, new vehicle categories, or extended analytics features after the initial requirements were frozen, expanding the project scope beyond what

	was planned.
Mitigation Monitoring	and A clear requirements document was agreed upon before development began. Any new requests are logged and deferred to a future version. Weekly progress reviews are held to ensure scope boundaries are maintained and no unplanned features are added during the current sprint.
Management	If scope change is unavoidable, an emergency review meeting will be held to assess its impact on the timeline and cost before accepting the change. Non-essential features will be deprioritised to accommodate the new requirement.
Status	Not Occur

Table 5.2: Project Risk 2

Name	Timeline Overrun Due to YOLO Model Retraining
Probability	Moderate (25–50%)
Impact	Tolerable
Description	YOLO11 model training requires multiple iterations to achieve acceptable detection accuracy on the custom car wash dataset. Each training cycle takes several hours, and poor evaluation results require re-annotation of images and retraining, which can cause significant delays.
Mitigation Monitoring	and A training schedule with defined accuracy milestones was established before training began. Early stopping and checkpoint saving minimise

	wasted GPU time. Six model variants (n/s/m/l/x sizes and one fine-tuned version) were evaluated in parallel to identify the best performer efficiently.
Management	If the timeline is exceeded due to model retraining, non-critical features such as advanced heatmap analytics will be deprioritised to protect the core detection and tracking delivery deadline. The project supervisor is informed immediately if the delay exceeds one week.
Status	Occurred, Managed

Technical Risks: The following technical risks have been identified for Smart Eye.

Table 5.3: Technical Risk 1

Name	GPU Hardware Failure / CUDA Unavailability
Probability	Low (10–25%)
Impact	Critical
Description	Smart Eye relies on a CUDA-enabled GPU for real-time YOLO inference at the required frame rate. Hardware failure, overheating, or an incompatible CUDA driver update would prevent the system from processing video at the required speed, making real-time monitoring impossible.
Mitigation and Monitoring	The system is designed with an automatic CPU fallback mode that activates when no CUDA device is detected. All dependency versions including PyTorch and CUDA are pinned in the requirements file to prevent unintended driver upgrades. GPU temperature and memory

	usage are monitored via a background health check.
Management	If the GPU fails during operation, the system automatically switches to CPU mode and increases the frame skip rate to maintain acceptable throughput. A spare machine with a compatible GPU is identified as a backup environment. The operator is notified via a prominent status banner.
Status	Not Occur

Table 5.4: Technical Risk 2

Name	Model Accuracy Degradation Under Poor Lighting or New Camera Angles
Probability	High (50–75%)
Impact	Tolerable
Description	The YOLO11 model trained on the custom dataset may produce low-confidence detections or miss vehicles entirely when environmental conditions change, for example during night-time operation, rainy weather with glare, or when the camera is repositioned to a different angle or height.
Mitigation and Monitoring	The training dataset was augmented with brightness, contrast, hue shifts, and rotation variations to improve generalisation. A dynamic low-light confidence adjustment module automatically lowers the detection threshold when the average frame brightness drops below a configurable value. Detection accuracy is reviewed periodically by replaying recorded sessions.

Management	If detection accuracy drops below an acceptable level in production, the operator is notified via the status panel. A supplementary dataset captured under the new conditions is collected and used to fine-tune the model. The updated model is deployed without interrupting ongoing sessions.
Status	Monitoring it

Table 5.5: Technical Risk 3

Name	Re-ID Failure Causing Duplicate or Split Session Records
Probability	Moderate (25–50%)
Impact	Tolerable
Description	When a vehicle temporarily leaves the camera frame and re-enters; for example when moving between the vacuum bay and the wash area, the Re-ID module may assign a new tracker ID instead of restoring the original, resulting in a duplicate entry record or a split wash session in the database.
Mitigation and Monitoring	The EnhancedFuelStationTracker combines colour histogram and HOG appearance features with position proximity to match lost tracks within a configurable timeout window. The IDMappingManager maintains a persistent entry-to-tracker mapping that survives short occlusions. Session records are reviewed periodically for anomalous durations or overlapping timestamps.
Management	If a duplicate record is detected, the operator can manually merge sessions using the History tab. A database integrity check is run to flag

	sessions with durations shorter than the minimum plausible wash time or with back-to-back entries for the same vehicle.
Status	Monitoring it

Table 5.6: Technical Risk 4

Name	Video Source Disconnection Crashing the Tracking Pipeline
Probability	Moderate (25–50%)
Impact	Catastrophic
Description	A sudden disconnection of the USB camera or RTSP video stream causes the VideoProcessingThread to lose its frame source, potentially corrupting an in-progress wash session record or leaving the database in an inconsistent state with an open entry and no corresponding exit.
Mitigation and Monitoring	The video capture loop checks the return value of every frame read operation. On failure, the thread emits a status_changed signal and pauses processing gracefully without crashing the application. SQLite WAL mode ensures that any partially written transaction is automatically rolled back. The GUI displays a prominent disconnection warning banner immediately.
Management	If disconnection persists for more than 30 seconds, the thread attempts to reconnect to the video source automatically. If reconnection fails, the operator is prompted to restart video capture from the Detection tab. Any wash session that was open at the time of disconnection is automatically marked with status = 'interrupted' to flag it for manual review.

Status	Not Occur
--------	-----------

Business Risks: The following business risks have been identified for Smart Eye.

Table 5.7: Business Risk 1

Name	Operator Resistance to Adopting the System
Probability	Low (10–25%)
Impact	Marginal
Description	Non-technical car wash staff may be reluctant to use the new monitoring system, preferring manual oversight. This could result in the system being underutilised, ignored during busy periods, or switched off entirely, defeating the purpose of automated session tracking.
Mitigation and Monitoring	The GUI was designed with a minimal learning curve; the Detection tab provides all core information on a single screen with no training required. A brief on-site demonstration session is planned before deployment. User feedback is collected informally during the first week of use to identify usability pain points.
Management	If adoption remains low after two weeks, a simplified one-page quick-start guide will be prepared and displayed near the monitoring workstation. Additional operator training sessions will be scheduled. If resistance continues, feedback will be incorporated into a future UI update.
Status	Not Occur

Table 5.8: Business Risk 2

Name	Third-Party Library Deprecation or Breaking Changes
Probability	Low (10–25%)
Impact	Marginal
Description	Smart Eye depends on several open-source libraries including Ultralytics YOLO, PyQt5, PyTorch, and OpenCV. Major version changes, API removals, or deprecation of any of these libraries could break core functionality and require significant refactoring to restore the system to a working state.
Mitigation and Monitoring	All dependency versions are pinned in a requirements.txt file to ensure reproducible builds. The project is tested against each pinned version before release. The changelogs for Ultralytics and PyTorch are reviewed quarterly. Deprecation warnings in the runtime log are monitored after each library update.
Management	If a critical dependency introduces a breaking change, the affected module is isolated and updated independently. The modular architecture of Smart Eye, with separate TrackingManager, FrameDetector, AreaTracker, and GUI layers, allows individual components to be upgraded without rewriting the entire system. A rollback to the last stable dependency set is available via the pinned requirements file.
Status	Not Occur

Chapter 6: Analysis & Modeling

Analysis modelling is a vital stage in software development that represents the specifications for data, function, and behaviour using a combination of text and diagrammatic representations. Its goal is to make the system's requirements plain and understood while facilitating the review process for accuracy, completeness, and consistency. Resources for traditional and object-oriented analytic techniques, as well as resources for the Unified Modelling Language (UML) (Booch, Rumbaugh and Jacobson, 2005), are introduced in this chapter. For Smart Eye, analysis modelling begins with the high-level system architecture, followed by the activity flows of all key operator interactions, the swim lane view of system responsibilities, the complete data flow from video input through to session reporting, and the entity relationship model of the database.

6.1 System Architecture Design

System architecture design defines the high-level structure of a software system, encompassing the components it consists of, how those components interact, and the principles that guide the organisation of the system as a whole. A well-defined architecture ensures that the system is maintainable, scalable, and understandable by all stakeholders involved in its development and operation.

Smart Eye follows a three-tier layered architecture in which each layer has a clearly defined responsibility and communicates only with the layers immediately adjacent to it. This separation of concerns reduces coupling between modules, simplifies testing, and makes it straightforward to replace or extend any individual component without affecting the rest of the system.

Table 6.1: System Architecture Layers of Smart Eye

Presentation Layer	PyQt5 MainWindow, Detection Tab, History Tab, Analytics Tab, Settings Tab	Renders the user interface, handles all operator interactions, and displays real-time video feed, live metrics, session history, analytics charts, and configuration
--------------------	---	--

		controls.
Processing Layer	YOLO11 Detection Engine, ByteTrack Tracker, Re-ID Module, IDMappingManager, Area Monitor, Alert Manager	Executes the core computer-vision pipeline, including frame capture, object detection, vehicle tracking with Re-ID, area boundary monitoring, session event generation, and overtime alert dispatch.
Data Layer	SQLite Database (WAL mode), JSON Configuration File, JPEG Snapshot Store	Persists all wash session records, entry and exit snapshot images, and operator configuration parameters. Provides query interfaces for session history, analytics aggregation, and configuration read/write operations.

The Presentation Layer communicates with the Processing Layer exclusively through Qt signals and slots, ensuring that the GUI thread is never blocked by heavy computation. The Processing Layer reads configuration from and writes session data to the Data Layer through well-defined database and file I/O interfaces, keeping the detection and tracking logic independent of any specific storage mechanism.

This layered design means that the YOLO model can be swapped, the database backend changed, or the GUI redesigned without requiring changes to the other layers, a property that significantly reduces the cost of future maintenance and extension.

6.2 Software Analysis Pattern

Software analysis patterns provide reusable solutions to recurring problems encountered during the analysis phase of software development. They help analysts structure their

understanding of the problem domain and translate requirements into a form suitable for design and implementation.

The objectives of software analysis patterns for Smart Eye are as follows:

1. Domain Analysis: Analysing the specific domain of automated car wash monitoring, in which the software will be used, in order to gain a deeper understanding of its operational requirements and constraints. This includes understanding how vehicles move through a car wash facility, what events are significant (entry, exit, overtime, park slot occupancy), and how the operator interacts with the monitoring system.

2. Describing Operator Requirements: Clearly describing the requirements and expectations of the operator as the sole end-user of Smart Eye. The operator needs real-time visibility of vehicle activity, automatic session logging without manual intervention, configurable detection zones, and access to historical session data and analytics at any time.

3. Establishing a Basis for Software Design: Providing a foundation for creating the software design by identifying key elements and relationships within the system. For Smart Eye this includes the relationships between the video processing pipeline, the tracking and Re-ID modules, the area monitoring logic, the SQLite session database, and the PyQt5 GUI layer.

4. Defining Valid Table Requirements: Specifying a set of criteria that can be verified and validated after the system is constructed. This includes verifiable thresholds such as detection confidence levels, Re-ID similarity scores, area transition frame counts, and overtime alert trigger times, all of which are configurable parameters that can be tested against expected system behaviour.

6.3 Activity Diagram

Graphical representations of processes comprising sequential activities and events within a system are called activity diagrams. They provide a visual depiction of options, iterations, and concurrency within a workflow. Both organisational and computational processes can employ activity diagrams. In the context of the Unified Modelling Language (UML) (Booch,

Rumbaugh and Jacobson, 2005), activity diagrams are used to represent the overall control flow inside a system.

Activity diagrams for Smart Eye are presented below, covering all major operator interactions and internal system processes. Each diagram uses standard UML notation: filled circles for start nodes, bull's-eye symbols for end nodes, rounded rectangles for activities, and diamonds for decision points.

6.3.1 Activity Diagram: System Startup

Figure 6.1 illustrates the system startup flow. The operator launches the application, which initialises the database connection and loads the configuration file. If no monitoring areas are defined, the operator is prompted to draw the wash zone and park slot boundaries before the YOLO model is loaded in a background thread and video processing begins.

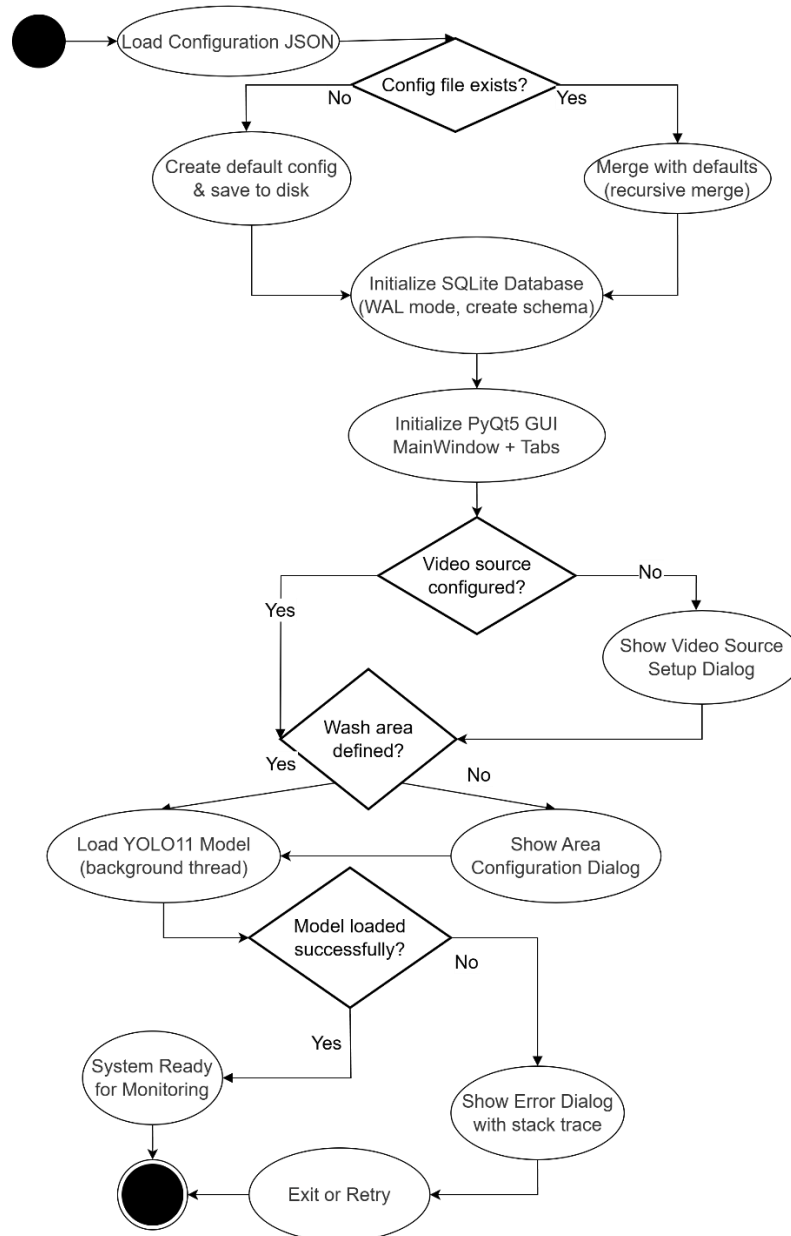


Figure 6.1: Activity Diagram: System Startup

6.3.2 Activity Diagram: Video Processing

Figure 6.2 shows the continuous video processing loop. Each frame read from the video source is evaluated to determine whether detection should run based on the configured frame skip rate. Detected vehicles are passed to the tracking module, area states are updated, events are logged, and the annotated frame is rendered to the Detection tab display.

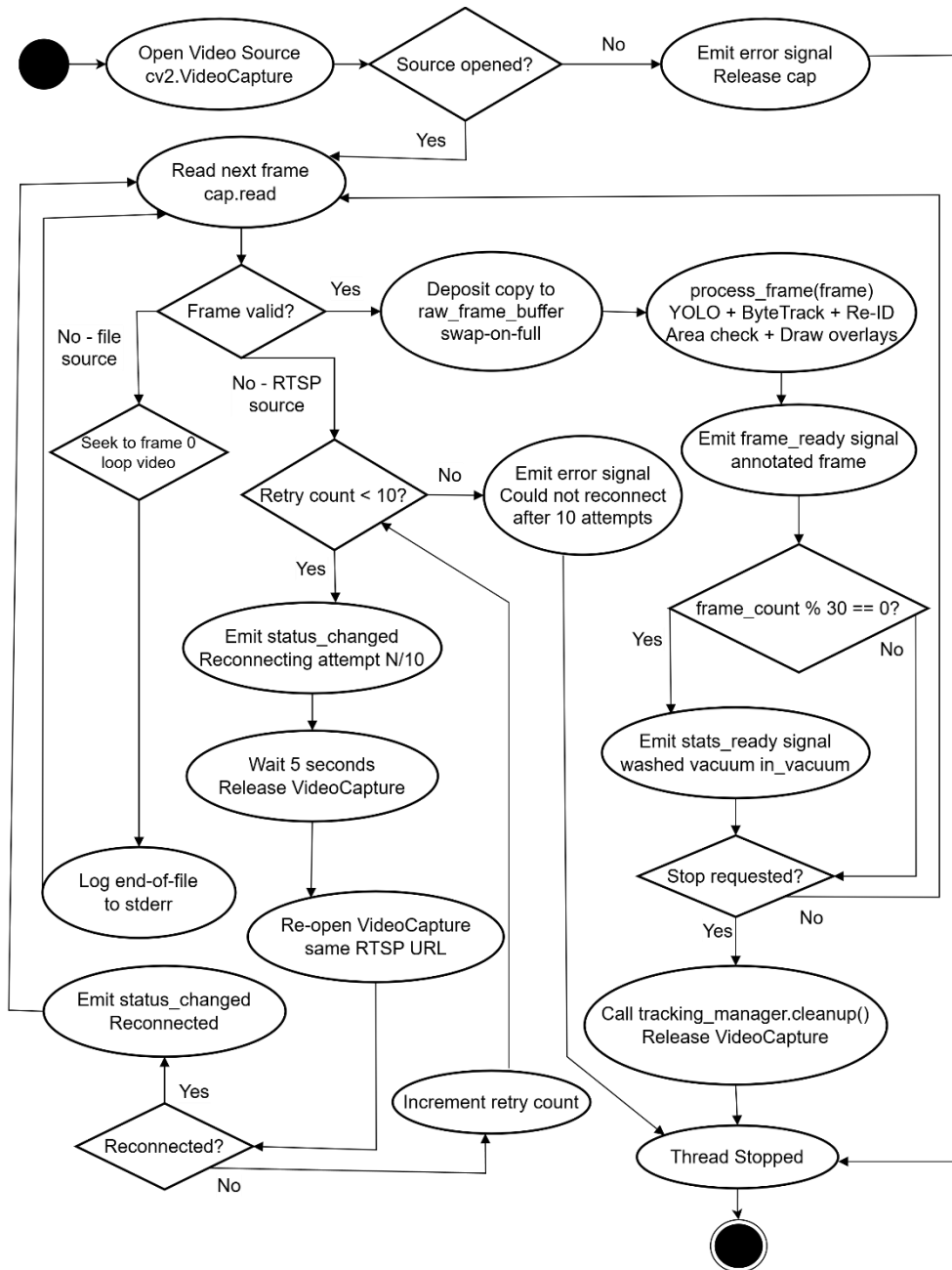


Figure 6.2: Activity Diagram: Video Processing Loop

6.3.3 Activity Diagram: Car Entry and Exit Detection

Figure 6.3 depicts the logic for detecting when a vehicle enters or exits the wash area. The system evaluates each tracked vehicle's centroid against the green rectangle boundary. A sustained presence inside the zone for a configurable number of frames triggers an entry event; sustained absence triggers an exit event and finalises the session record in the database.



Figure 6.3: Activity Diagram: Car Entry and Exit Detection

6.3.4 Activity Diagram: Alert Processing (Overtime)

Figure 6.4 presents the overtime alert processing flow. Once a vehicle has entered the wash area, the system continuously computes the elapsed time. When the duration exceeds the configured overtime threshold, a colour-coded alert banner is displayed in the Detection tab and the event is logged in the activity panel.

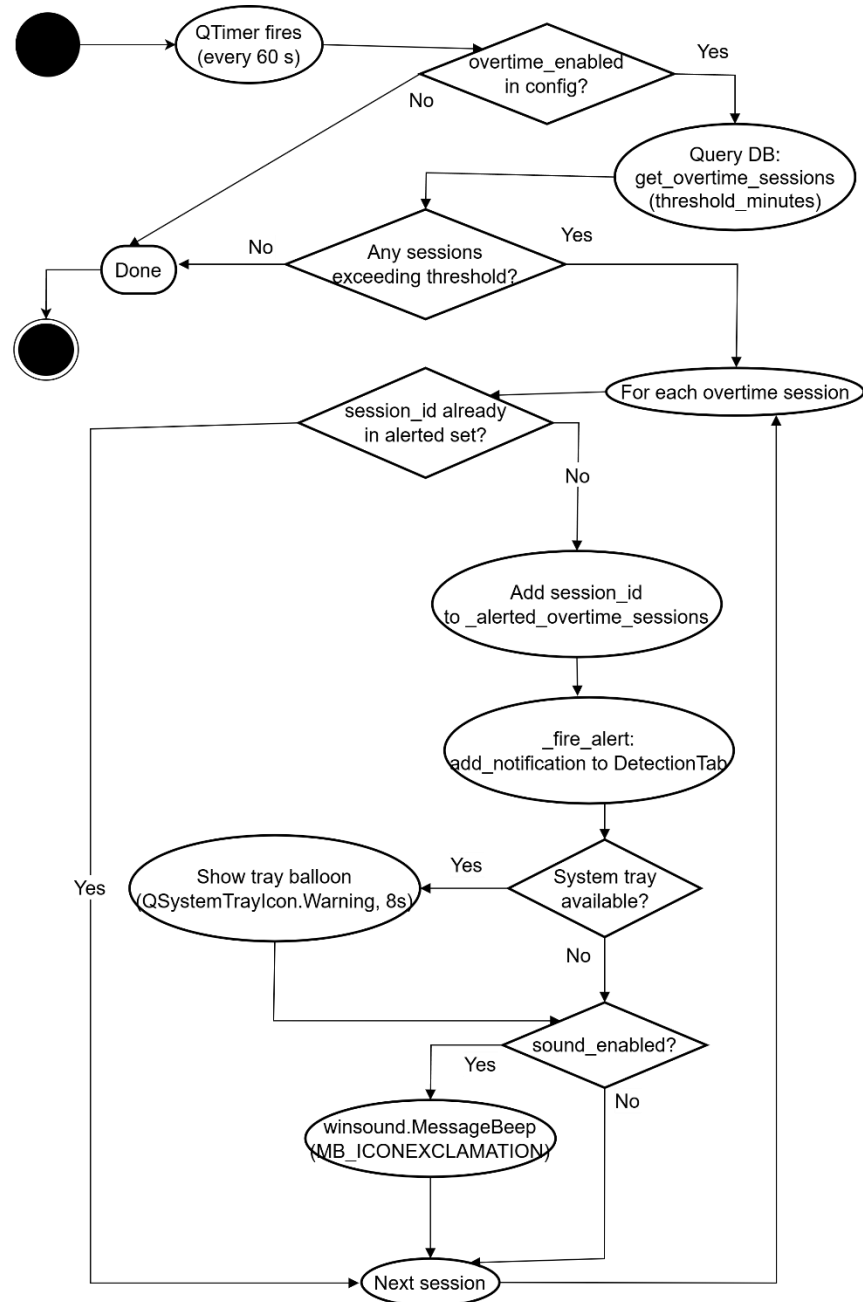


Figure 6.4: Activity Diagram: Alert Processing (Overtime)

6.3.5 Activity Diagram: Alert Processing (Vacuum Full)

Figure 6.5 illustrates the alert processing flow for vacuum bay occupancy. The system monitors the park slot boxes defined by the operator. When a vehicle occupies a slot beyond the permissible duration, a vacuum full alert is raised to notify the operator that the bay needs attention.

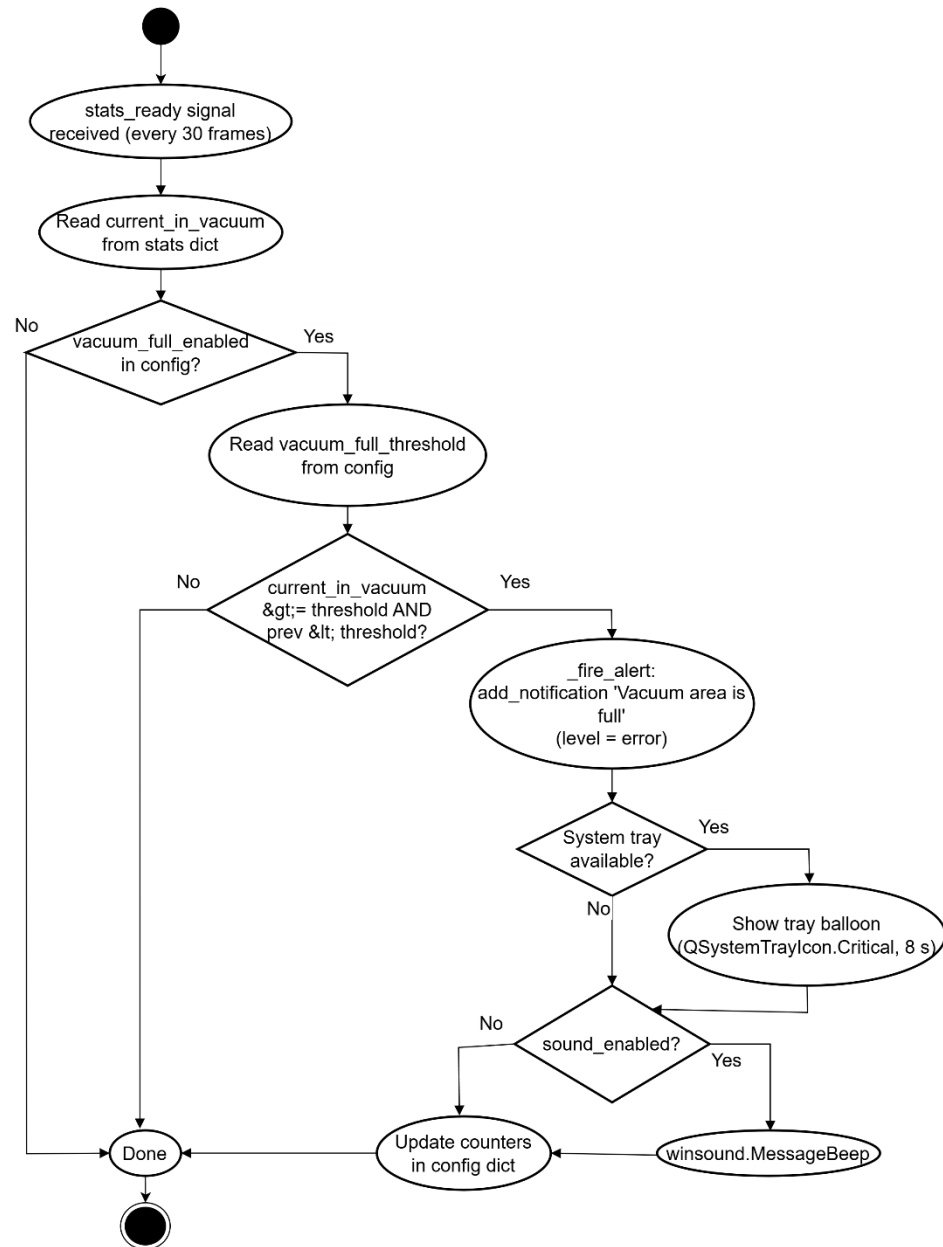


Figure 6.5: Activity Diagram: Alert Processing (Vacuum Full)

6.3.6 Activity Diagram: Session History Viewing

Figure 6.6 describes the flow for viewing and managing wash session history. The operator navigates to the History tab, where sessions are loaded from the SQLite database and displayed in a filterable table. The operator can search by date range or tracker ID, view entry and exit snapshot images, edit session notes, export records to CSV or PDF, and delete selected history entries.

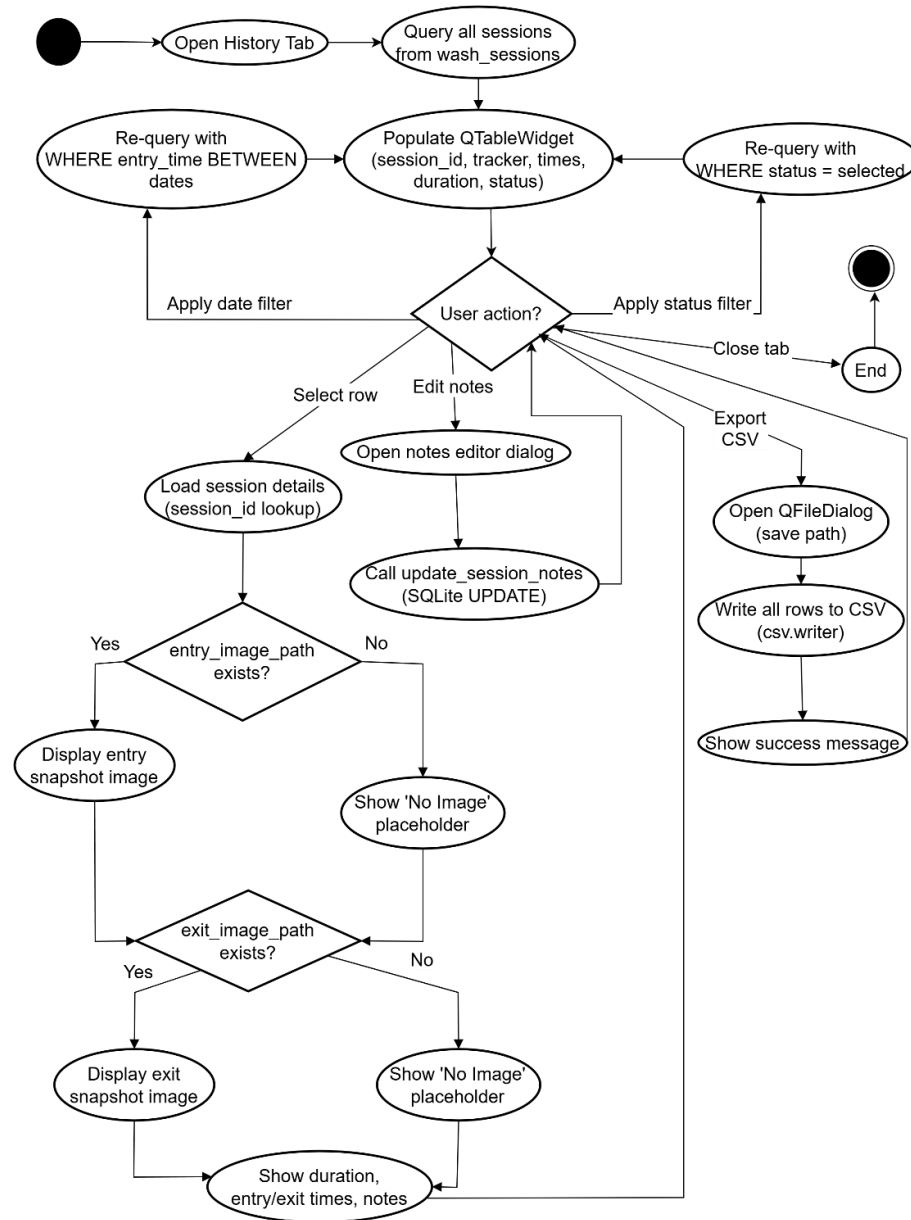


Figure 6.6: Activity Diagram: Session History Viewing

6.3.7 Activity Diagram: Area Configuration

Figure 6.7 shows the flow for configuring monitoring areas. The operator clicks Edit Areas to open the zone drawing dialog. Using mouse drag interactions, the operator draws the wash area rectangle and marks park slot boundaries. The coordinates are validated and saved to the JSON configuration file, taking effect immediately on the next processed frame.

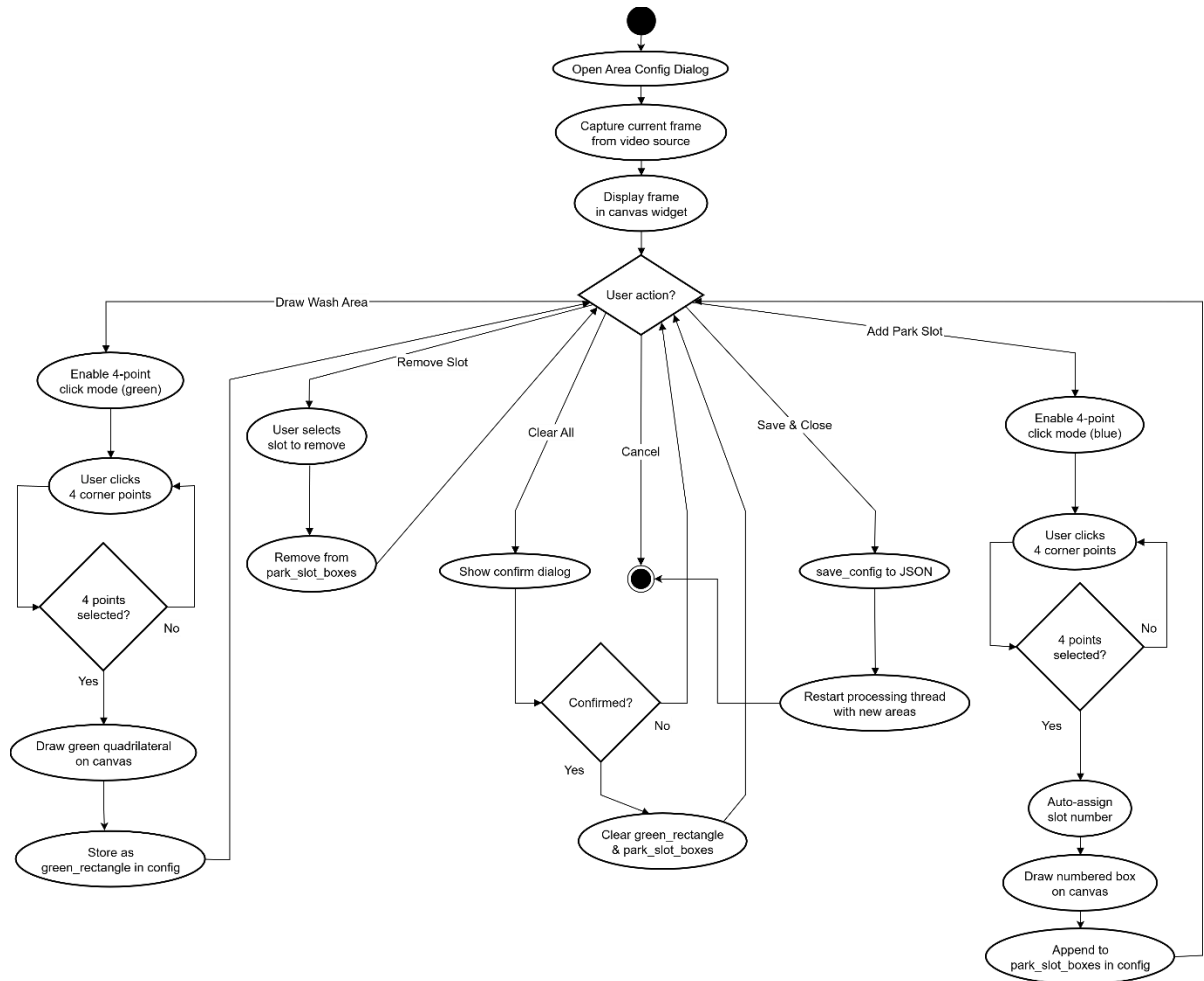


Figure 6.7: Activity Diagram: Area Configuration

6.3.8 Activity Diagram: Model Loading and Switching

Figure 6.8 illustrates the YOLO model loading and switching process. When the operator selects a new model file from the Settings tab, the current processing thread is paused and a background model loader thread is spawned. Once the new model is validated and loaded,

the detection thread resumes with the updated model without requiring a full application restart.

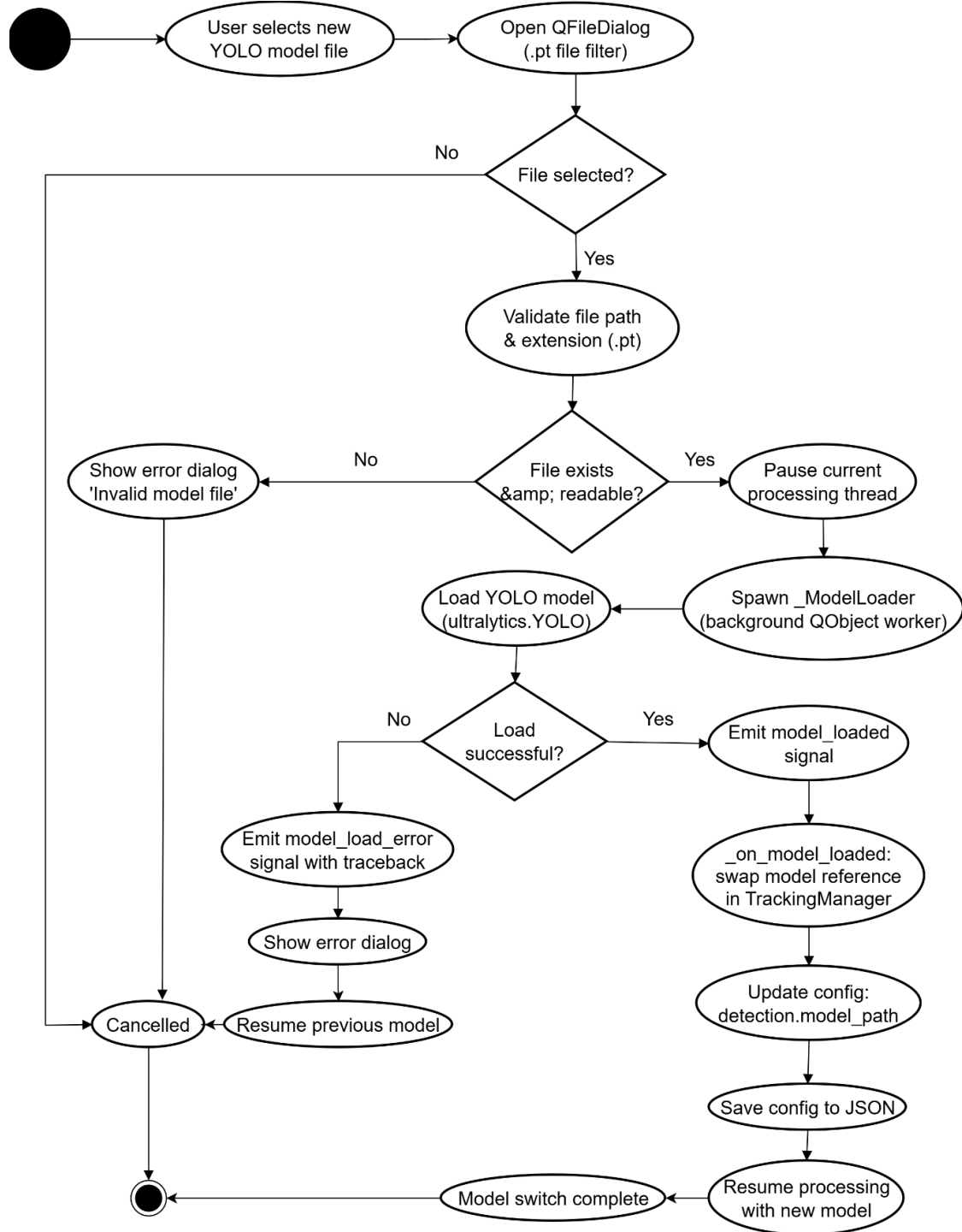


Figure 6.8: Activity Diagram: Model Loading and Switching

6.3.9 Activity Diagram: Settings Configuration

Figure 6.9 depicts the settings configuration flow. The operator adjusts parameters including video source type, RTSP URL or file path, confidence threshold, low-light confidence, Re-ID threshold, position threshold, frame skip rate, and overtime limit. On clicking Save Configuration, all values are validated and written to the JSON configuration file, and the active processing thread applies the new parameters.

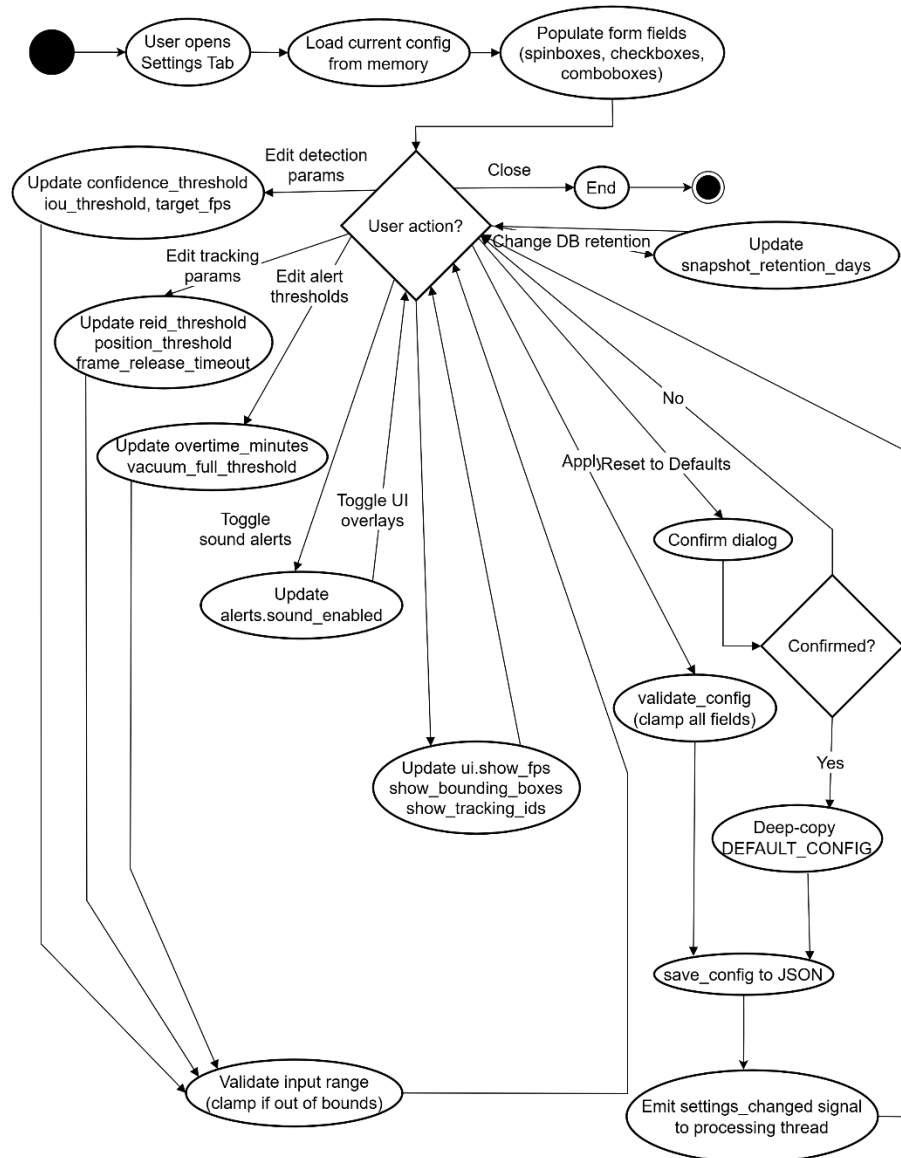


Figure 6.9: Activity Diagram: Settings Configuration

6.3.10 Activity Diagram: Analytics Dashboard

Figure 6.10 presents the analytics dashboard viewing flow. The operator navigates to the Analytics tab and selects a time range (last 7 days, 30 days, or custom). The system queries the wash_sessions database and renders three charts: a bar chart of cars washed per day, a line chart of average wash duration per day, and a peak hours heatmap showing session counts by hour of day.

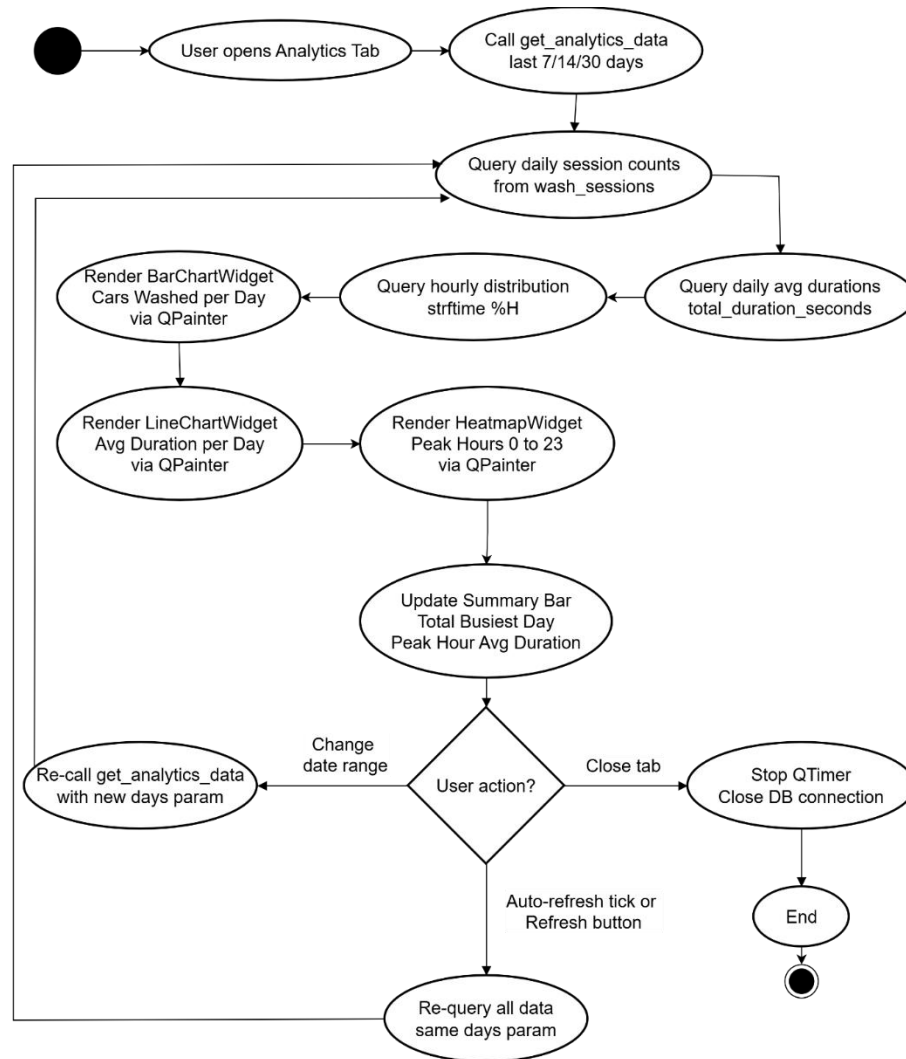


Figure 6.10: Activity Diagram: Analytics Dashboard

6.4 Swim Lane Diagram

A swim lane diagram is a visual representation that illustrates the flow of activities in a process while clearly indicating the responsible individuals, work groups, or system components involved. It takes its name from the lanes resembling separate swimming lanes in a pool, with each lane representing a specific participant or system component.

The swim lane diagram is used to bring clarity and accountability to a process by assigning process steps to their respective lanes. This allows for a clear understanding of which component is responsible for each activity. By visually mapping the process flow within the swim lanes, it becomes easier to identify communication channels and handoffs between different participants.

One of the key benefits of using a swim lane diagram is that it can help identify areas of redundancy and inefficiency within a process. By analysing the interactions between lanes and the sequence of activities, bottlenecks and areas for improvement can be identified more easily.

Figure 6.11 presents the swim lane diagram for Smart Eye, showing the interaction between the Operator and the System across all major workflows: starting video processing, detecting vehicle events, generating alerts, logging sessions, and viewing reports. The diagram makes clear which actions are initiated by the operator and which are performed autonomously by the system.

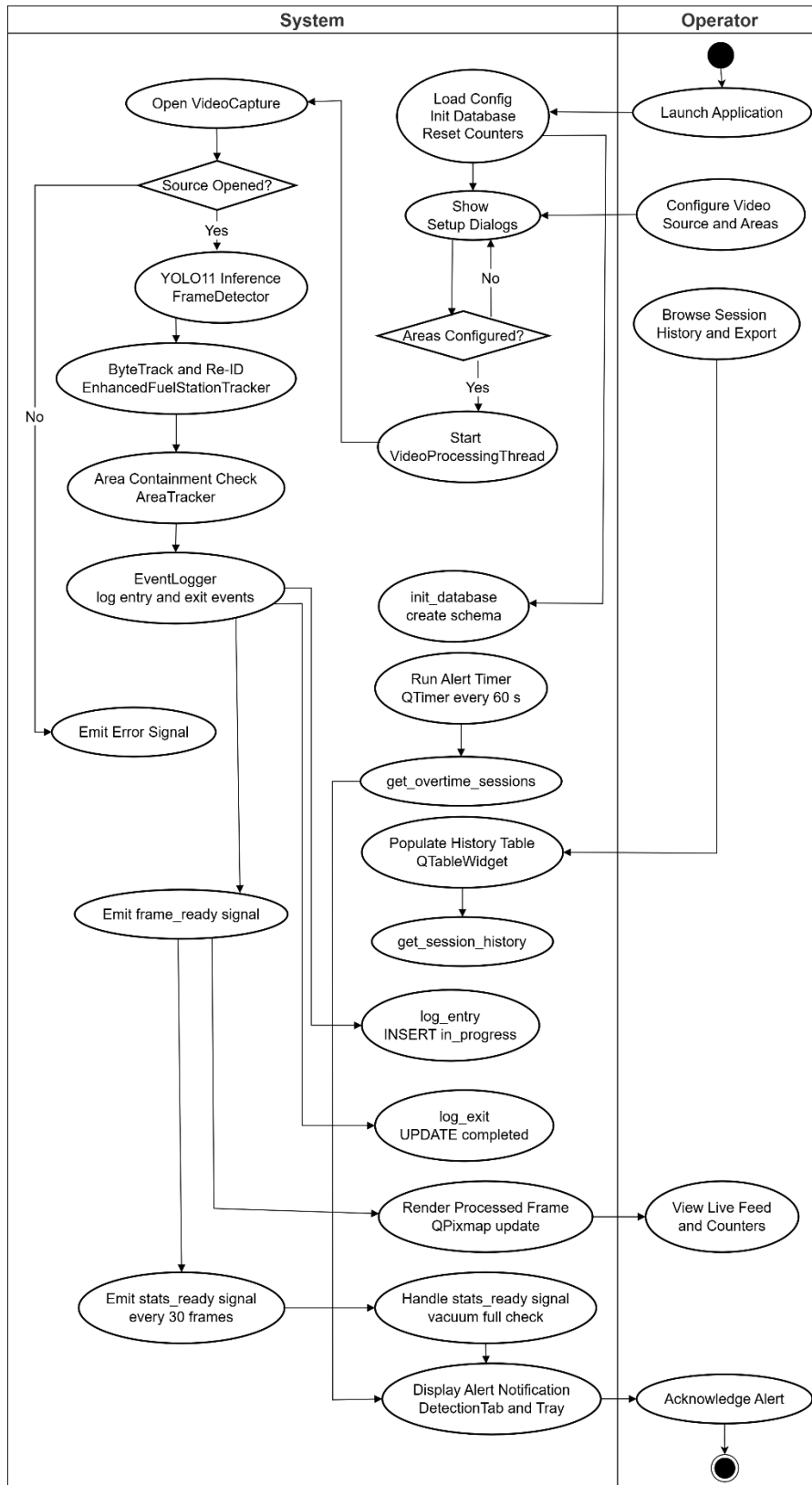


Figure 6.11: Swim Lane Diagram for Smart Eye

6.5 Data Flow Diagram

A Data Flow Diagram (DFD) (Pressman and Maxim, 2014; Sommerville, 2016) is a graphical representation that shows how information moves through a system or process. It offers a crystal-clear, organised perspective of how data flows across various system components and interactions. DFDs employ standardised symbols to depict elements and their interactions.

The essential elements of a DFD consist of:

1. Processes: Represented by rounded rectangles, processes depict the actions or activities that transform input data into output data.

2. Data Flows: Represented by arrows, data flows depict the movement of data from one component to another. They show how data is inputted, processed, and outputted within the system.

3. Data Stores: Represented by open-ended rectangles, data stores depict the places where data is stored or retrieved from. They can represent physical storage, databases, or other means of data storage.

4. External Entities: Represented by squares or rectangles, external entities are external sources or destinations of data that interact with the system. They can include users, hardware devices, or external data files.

The DFD illustrates the flow of data by showing how information enters and leaves the system, the transformations that occur within processes, and where data is stored. It provides a high-level overview of the system's functionality, allowing stakeholders to understand the interactions between different components. DFDs are commonly used in system analysis and design, requirements gathering, and process modelling.

6.5.1 Context Level DFD (Level 0)

Figure 6.12 presents the Context Level (Level 0) DFD for Smart Eye. At this level the entire system is represented as a single process. The external entities are the Operator (who provides configuration inputs and receives monitoring outputs) and the Video Source

(camera or video file that provides the raw video stream). The context diagram establishes the system boundary and identifies all external data flows entering and leaving Smart Eye.

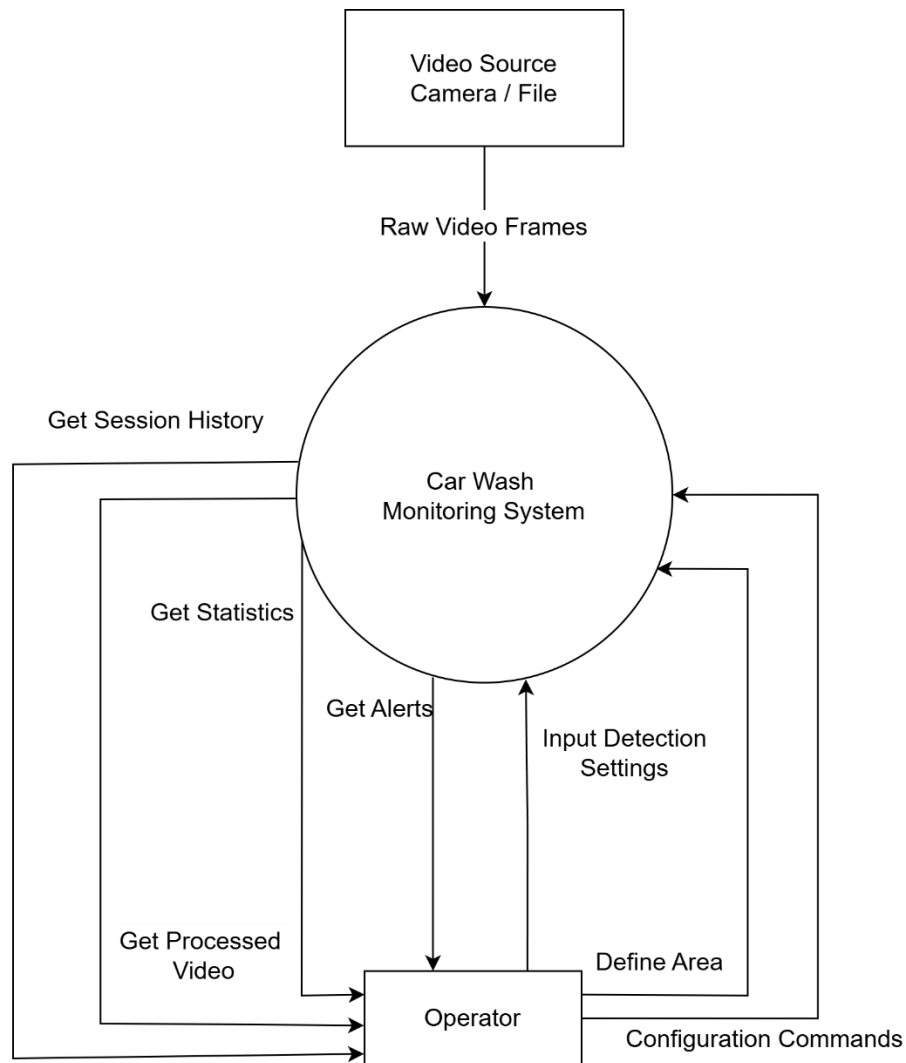


Figure 6.12: Context Level DFD (Level 0) for Smart Eye

6.5.2 Level 1 DFD

Figure 6.13 shows the Level 1 DFD, which expands the single system process into eight major sub-processes: P1 Video Capture, P2 Object Detection, P3 Vehicle Tracking, P4 Area Monitoring, P5 Session Logging, P6 Alert Management, P7 History and Analytics, and P8 Configuration Management. The diagram shows the primary data flows between these

processes and the three data stores: the Config File, the Car States in-memory store, and the Wash Sessions database.

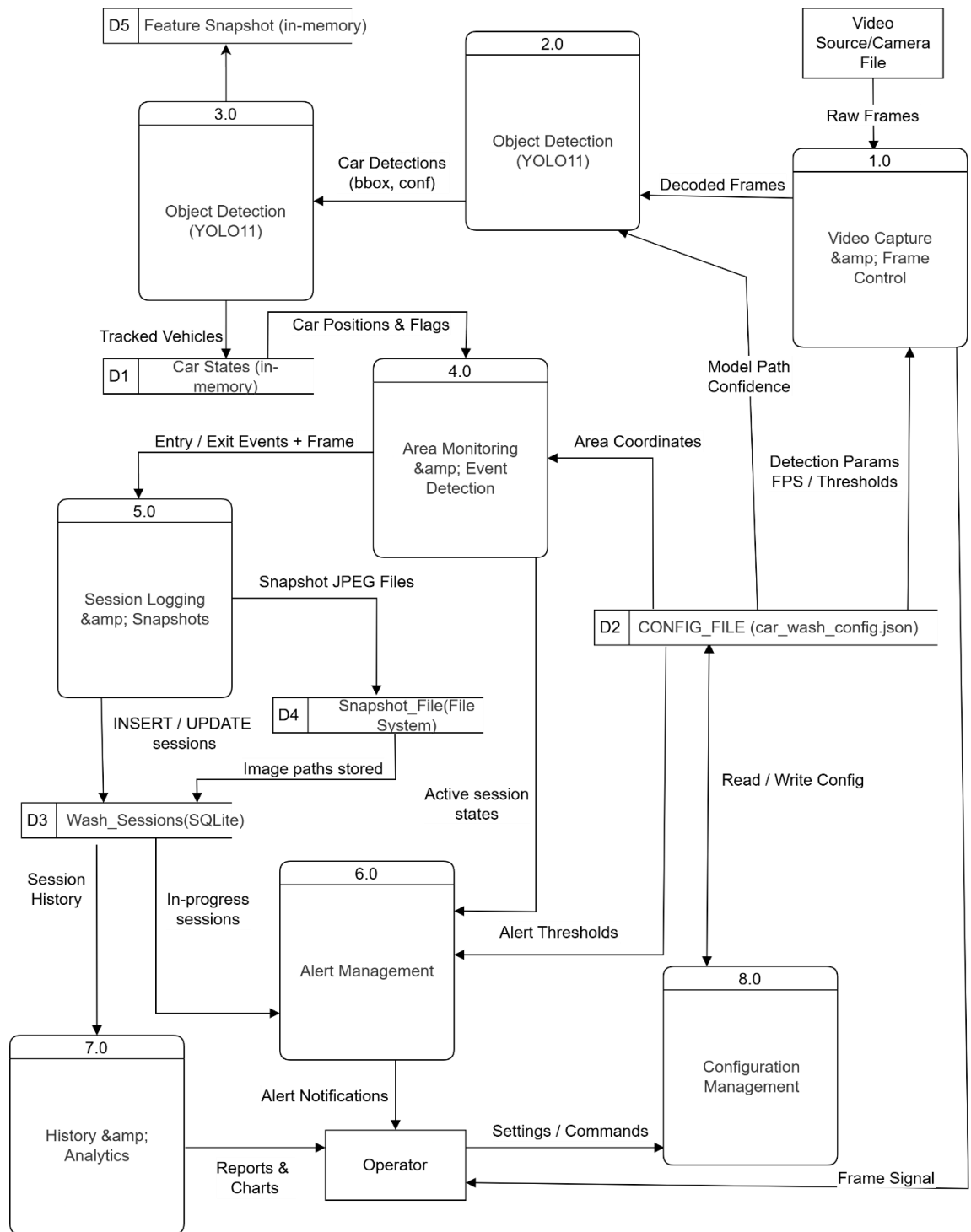


Figure 6.13: Level 1 DFD for Smart Eye

6.5.3 Level 2 DFD, Process 1: Video Capture

Figure 6.14 details Process 1 (Video Capture). This process reads frames from the configured video source, either a local video file or an RTSP camera stream, applies frame skipping according to the configured rate, and passes raw frames to the detection pipeline. It also handles source disconnection detection and reconnection attempts.

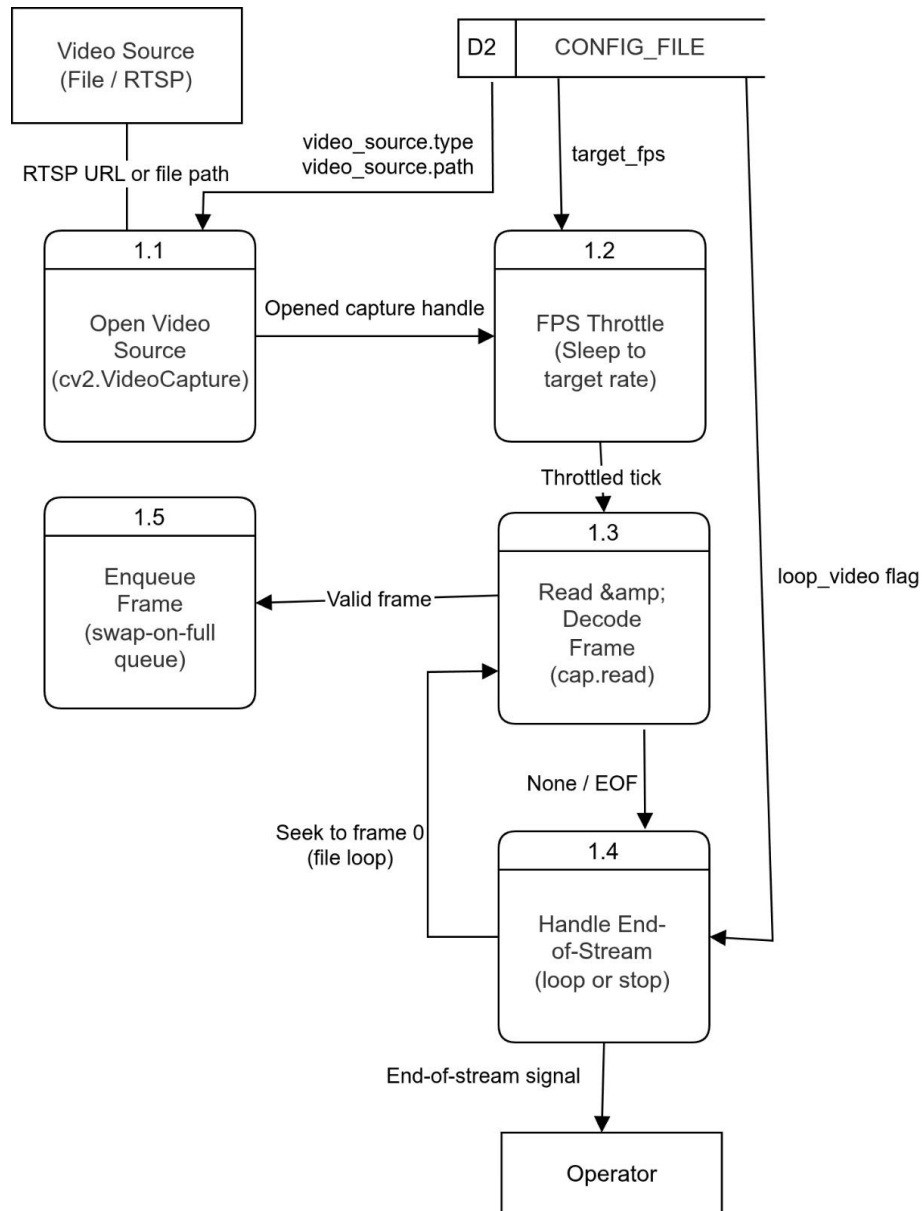


Figure 6.14: Level 2 DFD, Process 1: Video Capture

6.5.4 Level 2 DFD, Process 2: Object Detection

Figure 6.15 details Process 2 (Object Detection). This process receives raw frames and runs YOLO11 inference to produce bounding box detections. It applies dynamic confidence thresholds, using the standard threshold under normal conditions and the lower low-light threshold when frame brightness falls below a configurable level, and passes validated detections to the tracking module.

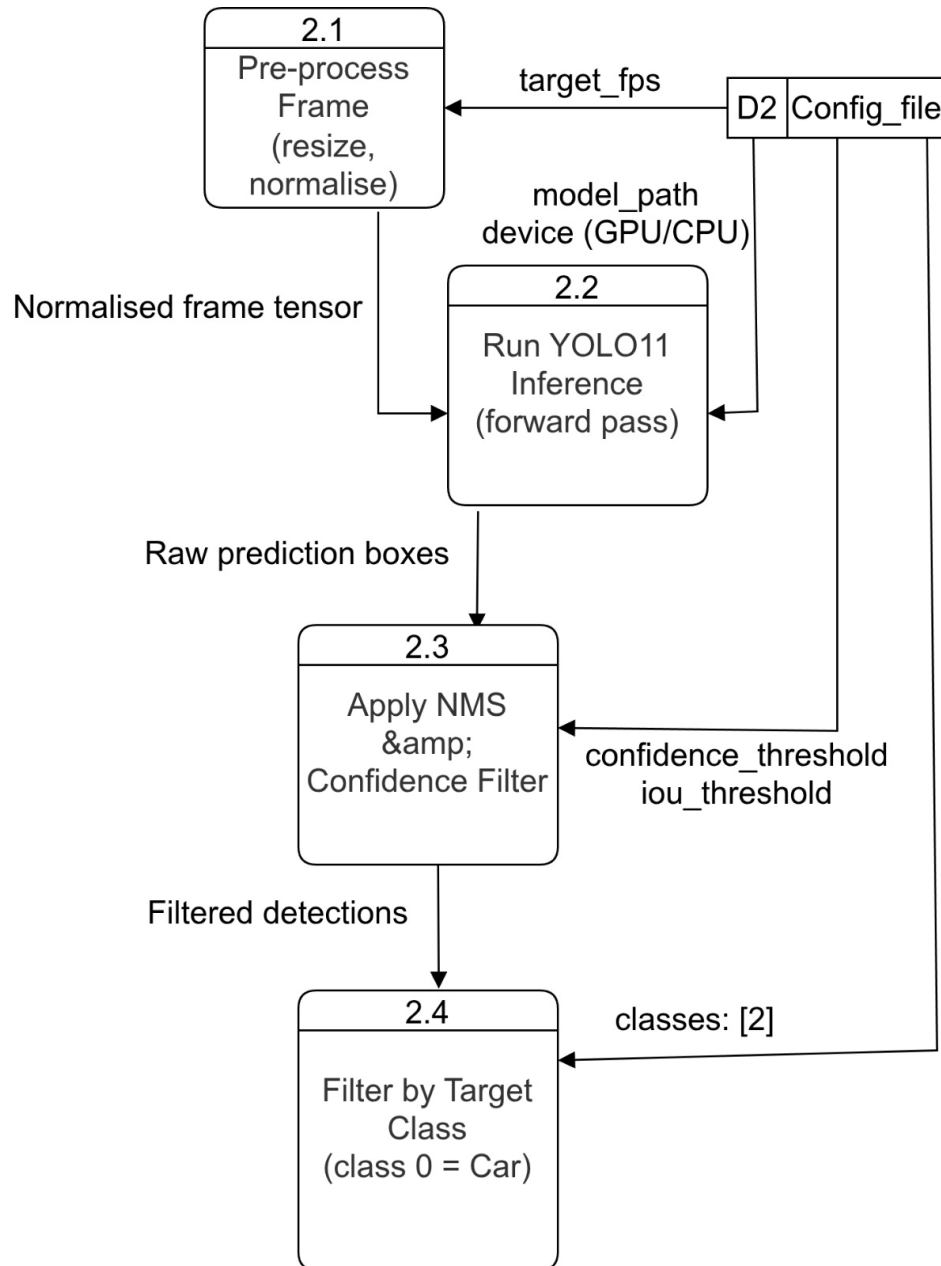


Figure 6.15: Level 2 DFD, Process 2: Object Detection

6.5.5 Level 2 DFD, Process 3: Vehicle Tracking

Figure 6.16 details Process 3 (Vehicle Tracking). This process maintains persistent tracker IDs across frames using ByteTrack (Zhang et al., 2022). When a track is lost, the Re-ID module (Ye et al., 2022) computes an appearance feature vector and the IDMappingManager attempts to match it to a previously seen vehicle within the configured timeout window, preserving session continuity across short occlusions.

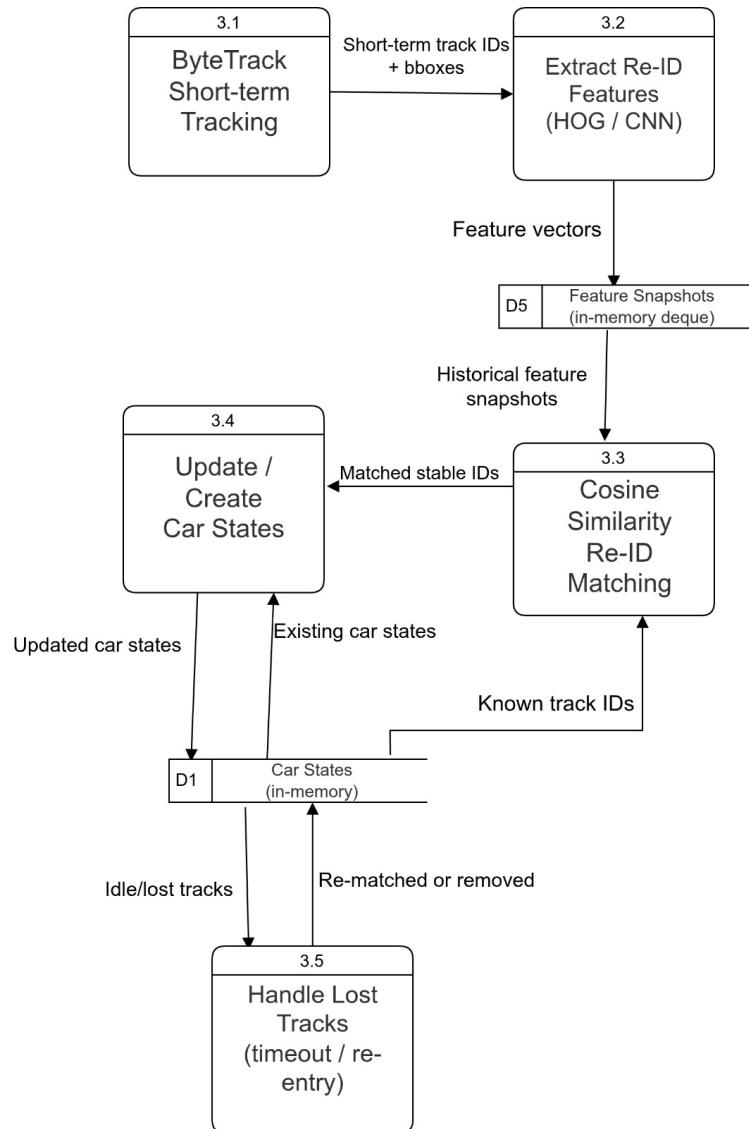


Figure 6.16: Level 2 DFD, Process 3: Vehicle Tracking

6.5.6 Level 2 DFD, Process 4: Area Monitoring

Figure 6.17 details Process 4 (Area Monitoring). This process loads the wash area rectangle and park slot coordinates from the configuration file and performs point-in-polygon tests for each active vehicle centroid. It detects entry and exit transitions for the wash zone and occupancy changes for each park slot, updating the in-memory car states accordingly.

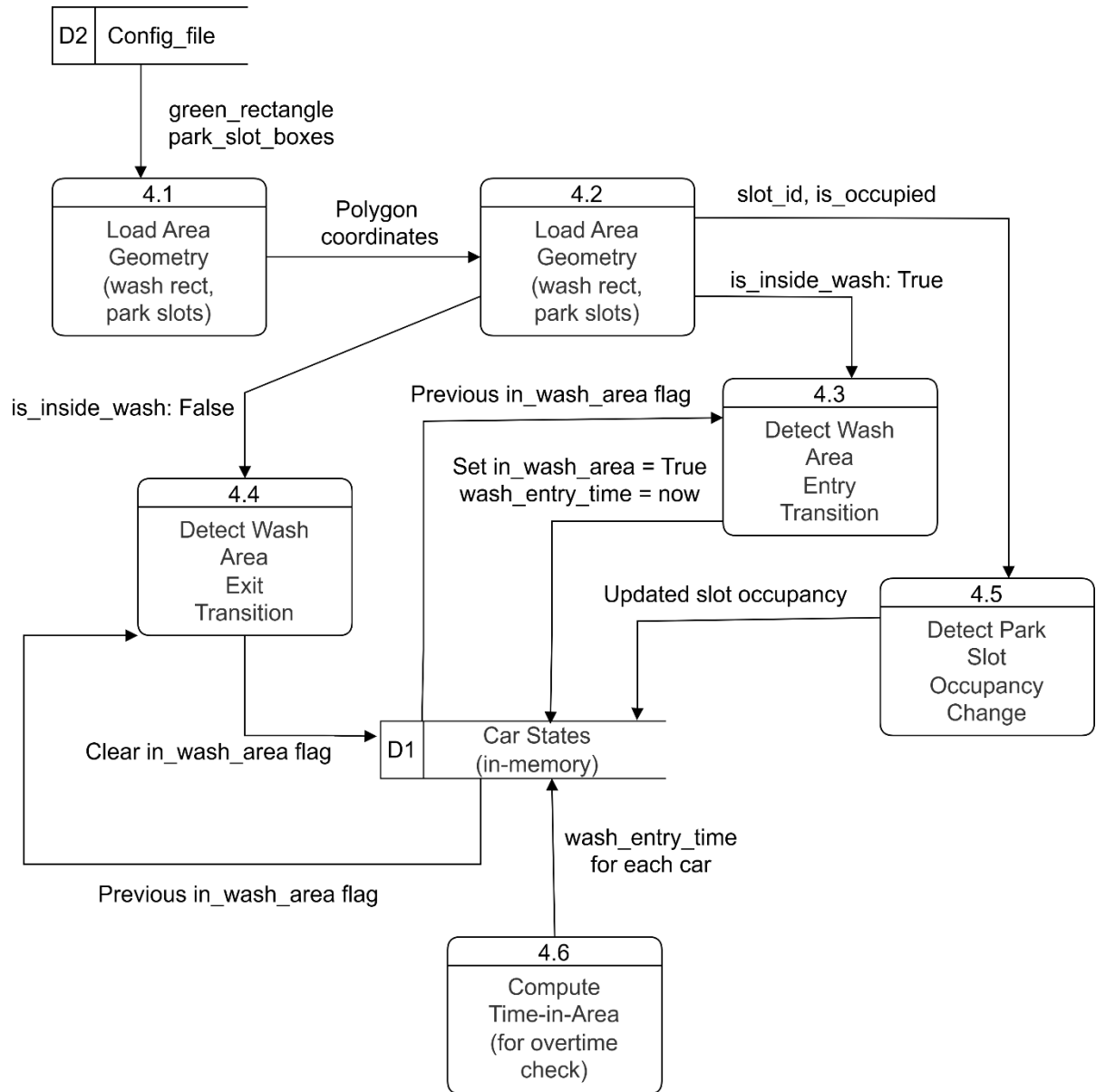


Figure 6.17: Level 2 DFD, Process 4: Area Monitoring

6.5.7 Level 2 DFD, Process 5: Session Logging

Figure 6.18 details Process 5 (Session Logging). On receiving an entry event, this process creates a new record in the wash_sessions SQLite table and captures a JPEG entry snapshot. On receiving an exit event, it updates the record with the exit time, total duration, exit snapshot path, and final session status. SQLite WAL mode (Riverbed Technology, no date) ensures all writes are atomic and crash-safe.

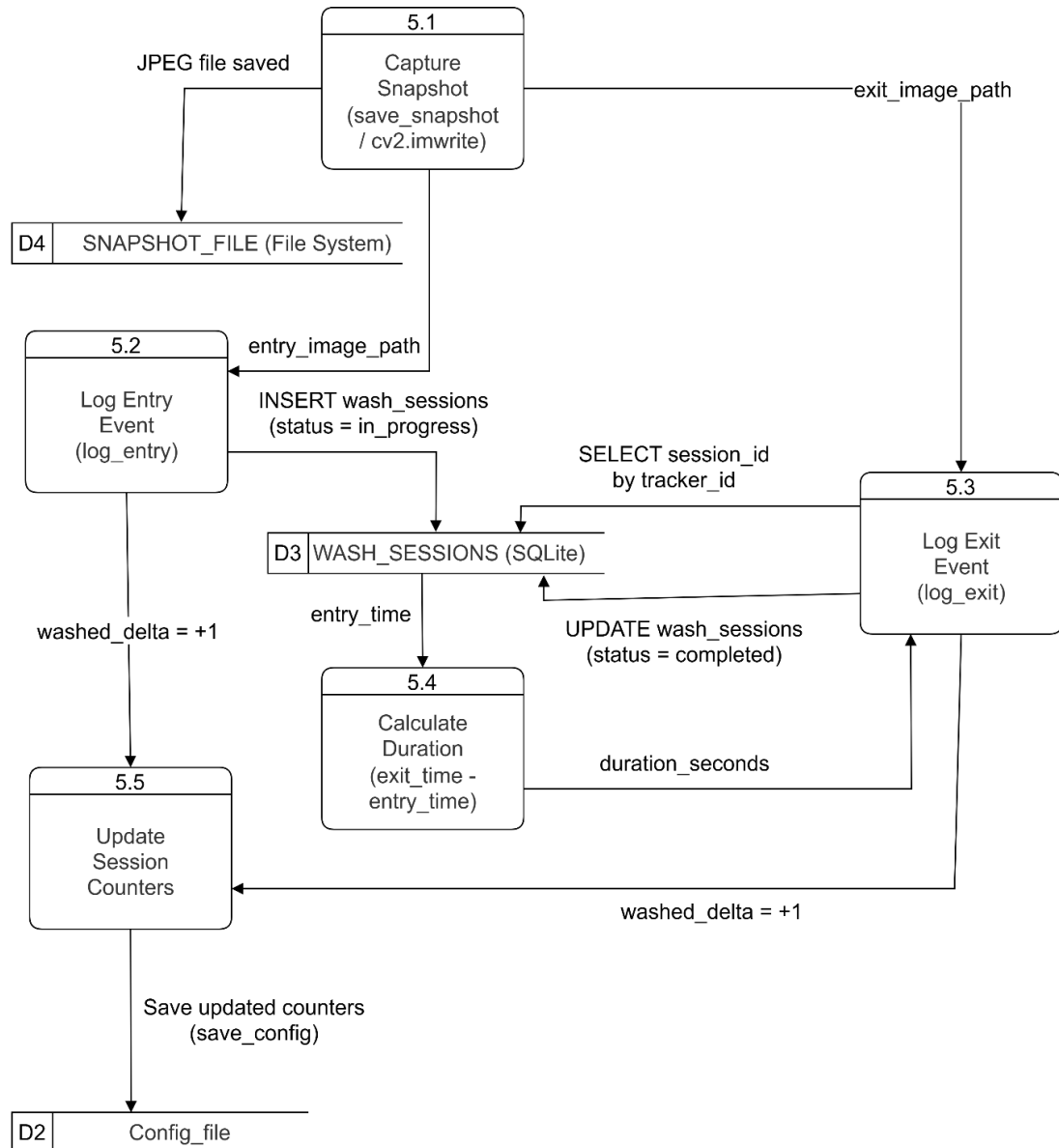


Figure 6.18: Level 2 DFD, Process 5: Session Logging

6.5.8 Level 2 DFD, Process 6: Alert Management

Figure 6.19 details Process 6 (Alert Management). This process continuously monitors active wash sessions for overtime conditions by comparing the elapsed time since entry against the configured overtime limit. When a threshold is breached, it emits an alert signal to the GUI, which displays a colour-coded banner in the Detection tab and logs the alert in the activity panel.

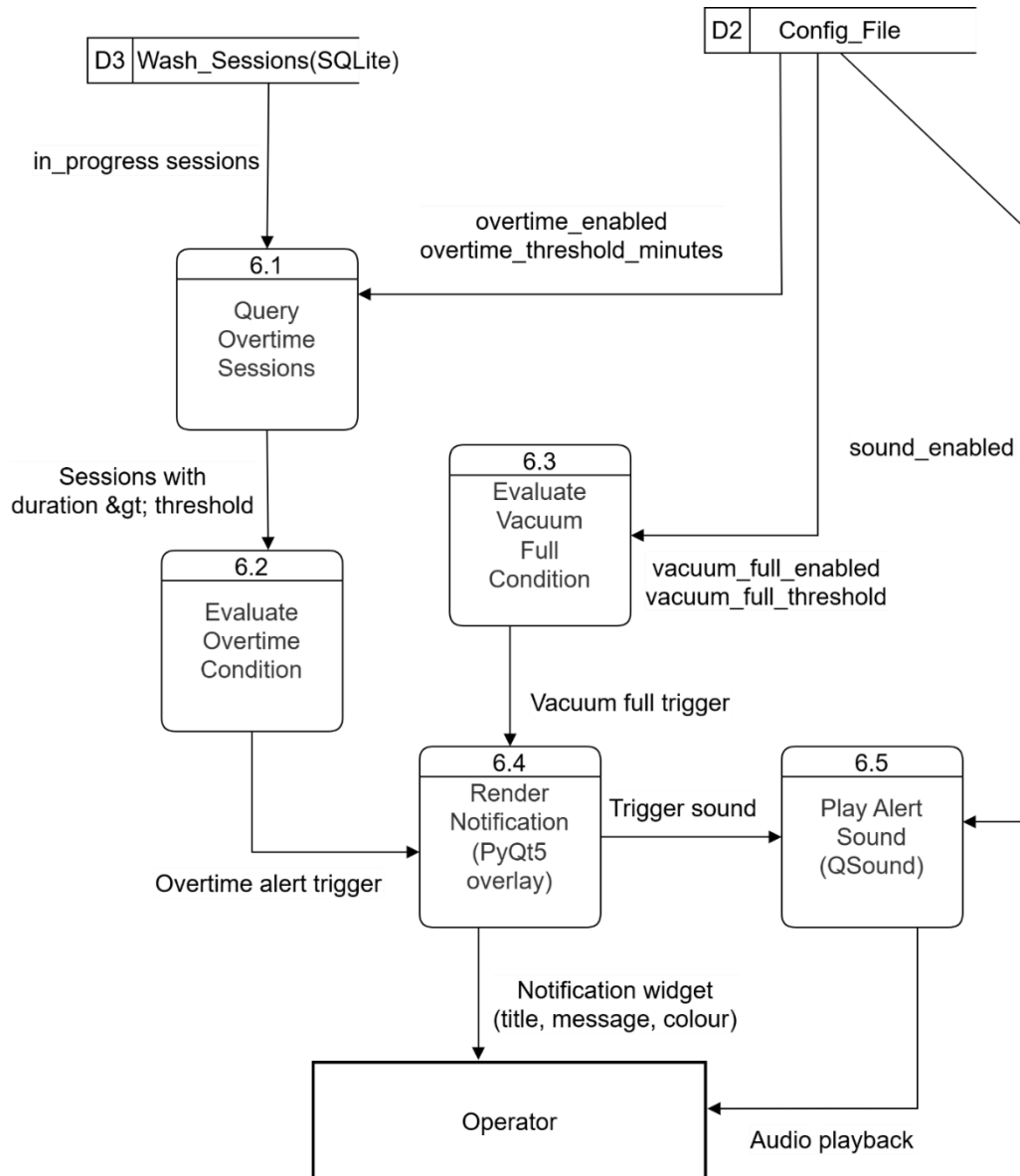


Figure 6.19: Level 2 DFD, Process 6: Alert Management

6.5.9 Level 2 DFD, Process 7: History and Analytics

Figure 6.20 details Process 7 (History and Analytics). This process queries the wash_sessions database in response to operator requests from the History and Analytics tabs. It supports filtering by date range and session status, renders summary statistics, generates bar charts, line charts, and heatmaps from the aggregated session data, and supports export to CSV and PDF formats.

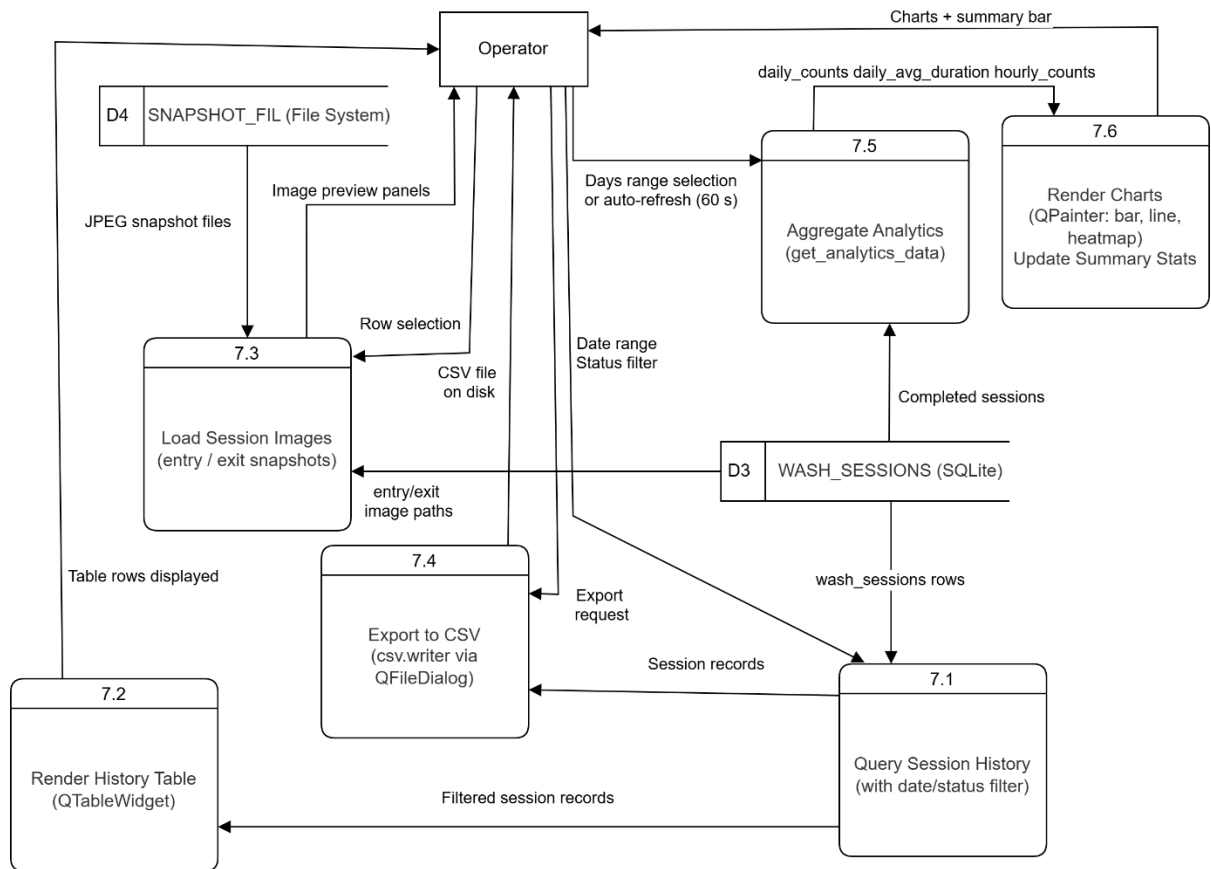


Figure 6.20: Level 2 DFD, Process 7: History and Analytics

6.5.10 Level 2 DFD, Process 8: Configuration Management

Figure 6.21 details Process 8 (Configuration Management). This process reads and writes the JSON configuration file in response to operator inputs from the Settings tab and the area drawing dialog. It validates all parameter values before persisting them and propagates changes to the active processing thread so that new settings take effect without restarting the application.

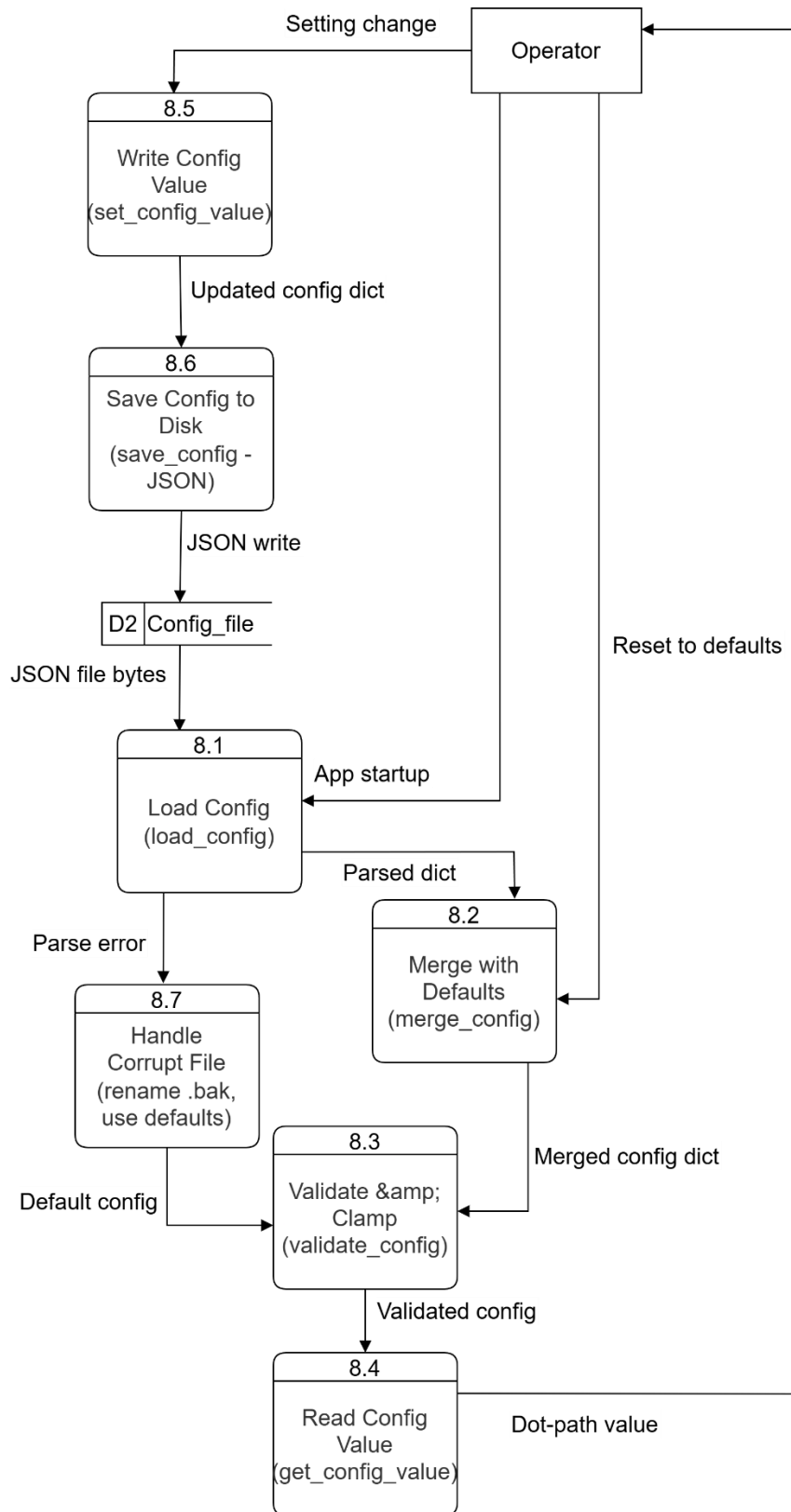


Figure 6.21: Level 2 DFD, Process 8: Configuration Management

6.6 Entity Relationship Diagram

An Entity Relationship Diagram (ERD) (Date, 2003) is a visual representation of the data model of a system. It illustrates the entities within the system, the attributes that describe each entity, and the relationships between entities. ERDs are widely used in database design to communicate the logical structure of data before physical tables are created.

For Smart Eye, the data model is deliberately simple: all persistent state is captured in a single central entity, the wash session, supplemented by configuration data stored externally in a JSON file. This design minimises database complexity while ensuring that every vehicle event is fully traceable from entry snapshot through to exit record.

Figure 6.22 presents the Entity Relationship Diagram for Smart Eye. The diagram shows the WashSession entity with its attributes (session_id, tracker_id, entry_time, exit_time, duration, status, entry_image_path, exit_image_path, notes), its relationship to the AreaConfiguration entity (which defines the wash zone and park slot coordinates), and its association with the SnapshotFile entity representing the captured JPEG images stored on disk.

Chapter 7: Designing

The designing phase translates the analysis and modelling artefacts into a concrete user interface that the operator interacts with directly. Smart Eye implements a dark-themed desktop application built with PyQt5, organised into four main tabs: Detection, History, Analytics, and Settings. The following figures present the implemented interface screens of the system.

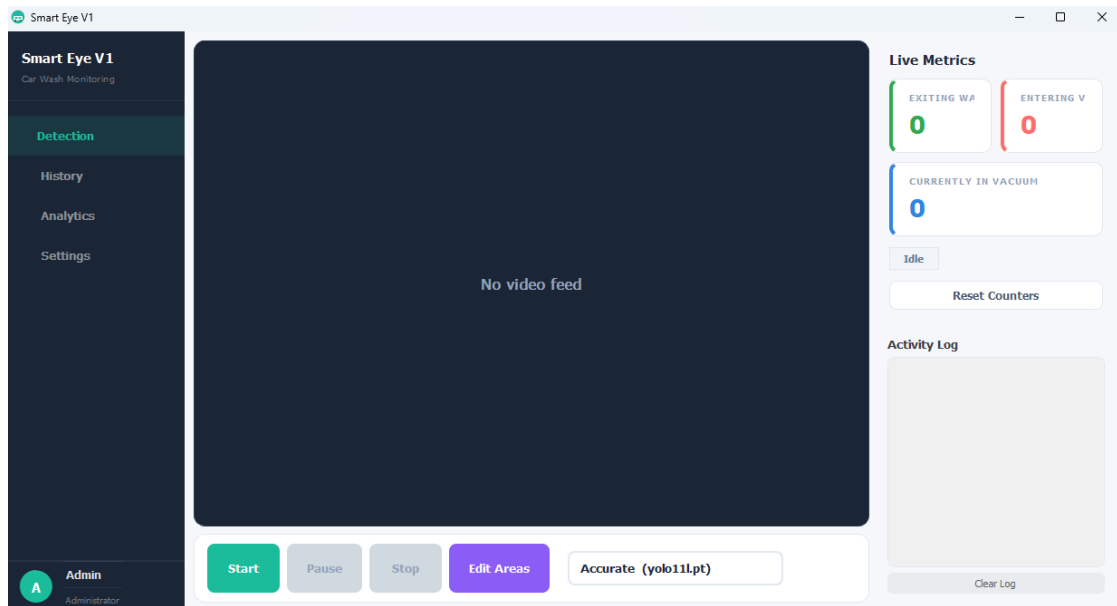


Figure 7.1: Detection Tab, Idle State

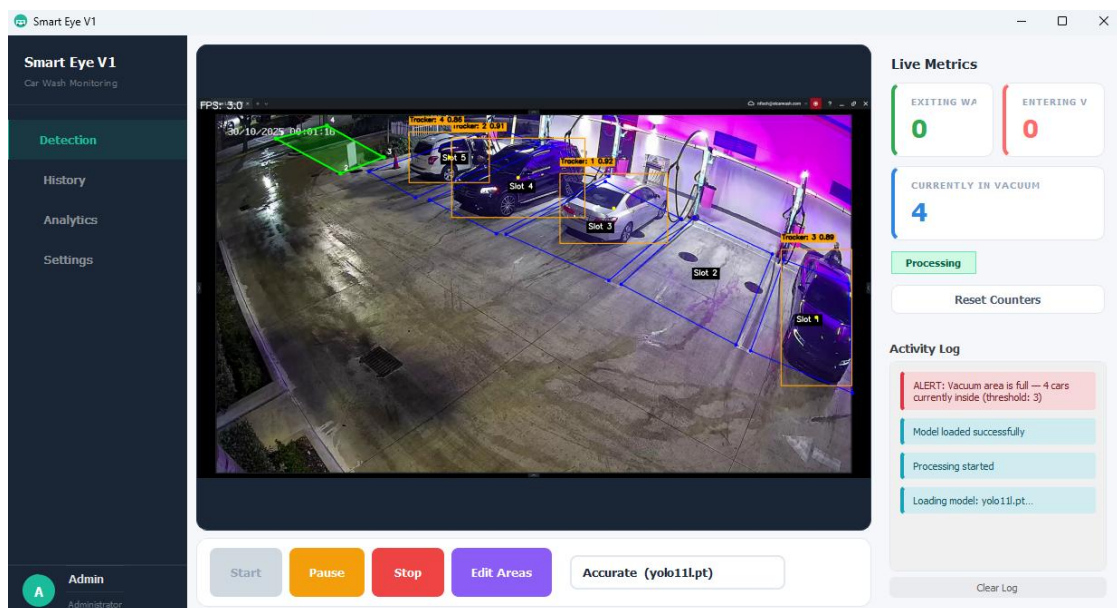
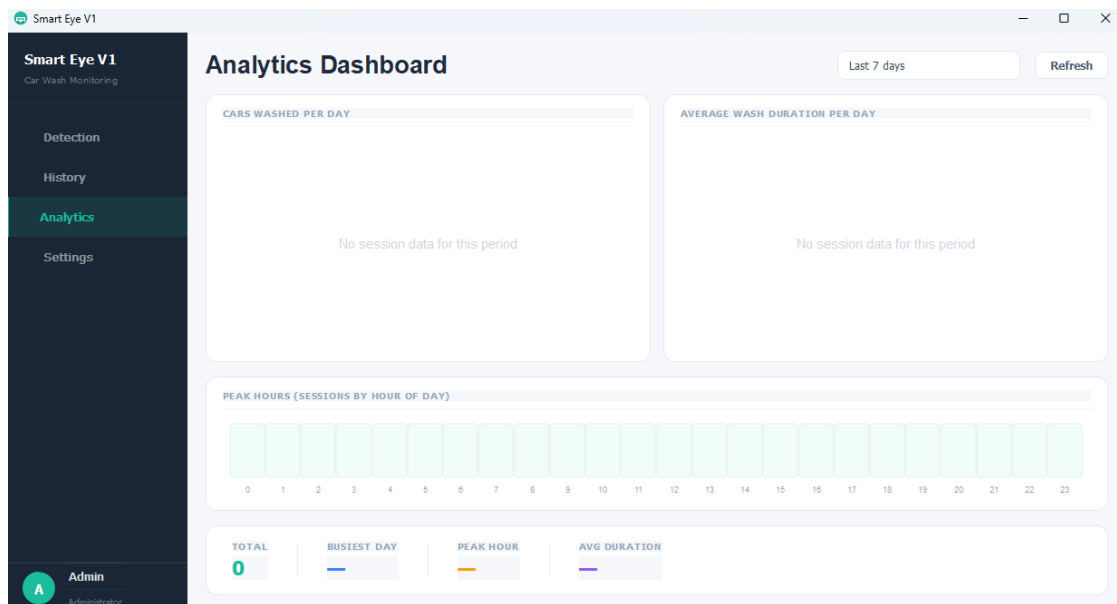
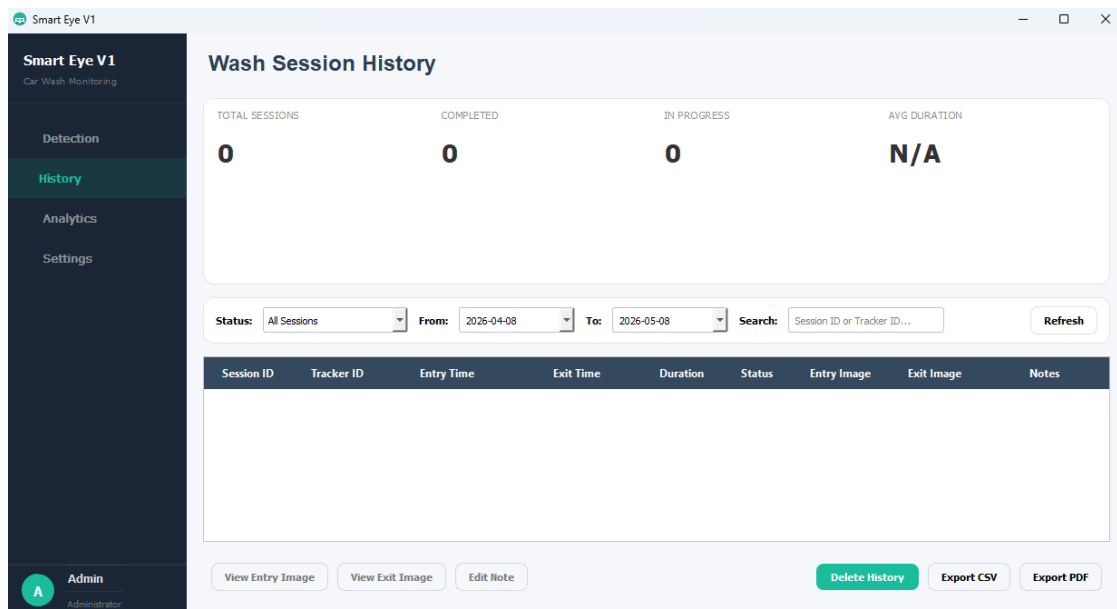


Figure 7.2: Detection Tab, Active State



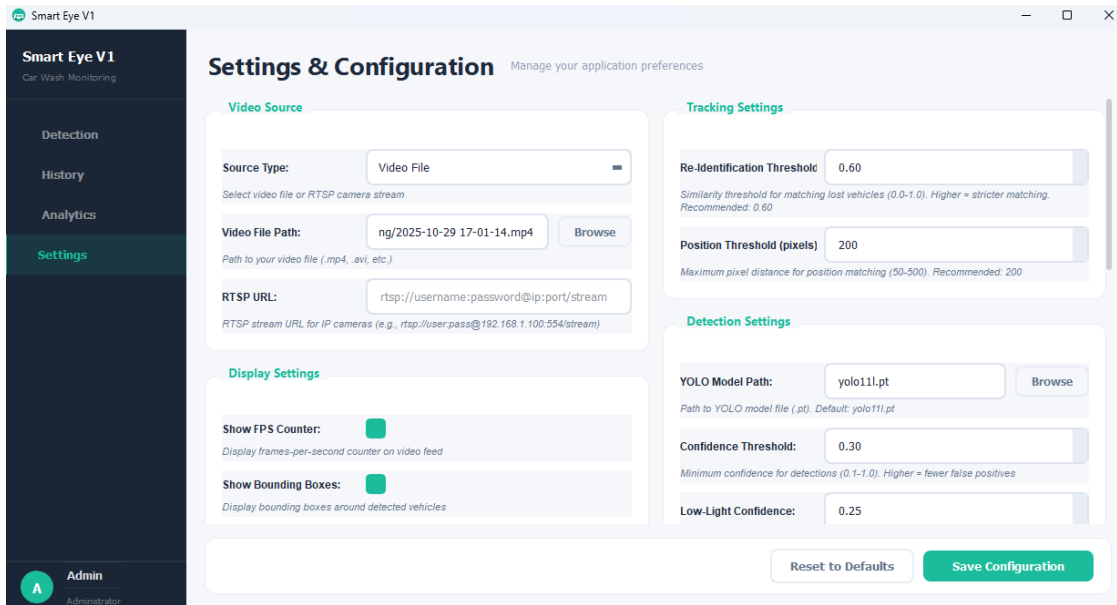


Figure 7.5: Settings and Configuration

Chapter 8: System Quality Management

Software testing is a critical step in the creation of software that entails assessing a software component to find any discrepancies between the intended output and the output that the software actually produces (Pressman and Maxim, 2014; Sommerville, 2016). It seeks to evaluate the product's qualities and characteristics to make sure that it satisfies the requirements and performs as planned. For Smart Eye, quality management ensures that the system reliably detects and tracks vehicles, correctly logs session data, and delivers a responsive and usable interface to the car wash operator under all expected operating conditions.

Testing serves as a means of verification and validation:

Verification: It entails determining if the product complies with the requirements established at the start of the development process. It emphasises verifying that the item operates as planned and adheres to the required standards. For Smart Eye, verification confirms that the detection confidence thresholds, Re-ID similarity scores, area transition frame counts, session logging logic, and overtime alert timings all behave in accordance with the specified requirements documented in the system analysis chapter.

Validation: The product is developed in accordance with the operator's expectations and satisfies the defined standards. It entails assessing the finished item to see if it corresponds with the anticipated usage and functioning. For Smart Eye V1, validation confirms that the operator can monitor vehicle activity in real time, review complete session histories, export reports, and adjust system configuration, all without requiring technical knowledge of the underlying computer vision pipeline.

8.1 System Quality Management

System Quality Management is essential for maintaining a software system's overall quality. It entails putting in place a quality management framework that evaluates and links every component of a system that is subject to quality standards. A Total Quality Management (TQM) approach or Quality Management System (QMS) makes it easier for the

development team to measure, communicate, and continuously improve the quality of the product at various stages of its development lifecycle.

There are two main types of quality that are important in software development:

Internal Quality:

- **Test Coverage:** The extent to which Smart Eye has been tested to ensure that all functional pathways are exercised. This includes testing the vehicle entry and exit detection logic, the Re-ID matching pipeline, the overtime alert trigger conditions, session record creation and finalisation, and all configuration read and write operations.
- **Testability:** The ease with which the components of Smart Eye can be tested individually. The three-tier layered architecture, comprising the Presentation, Processing, and Data layers, promotes testability by allowing the detection engine, tracker, and database manager to be exercised independently of the GUI layer.
- **Portability:** The ability of Smart Eye to be transferred or adapted to different environments. The system is developed in Python with cross-platform libraries (PyQt5, OpenCV, SQLite), enabling deployment on Windows and Linux machines without modifications to the core codebase.
- **Thread-Safeness:** The ability of the system to handle multiple concurrent threads without data corruption or race conditions. Smart Eye runs three concurrent threads: the PyQt5 GUI thread, the video processing and detection thread, and the background model loader thread, with all inter-thread communication handled exclusively through Qt signals and slots, ensuring thread-safe data exchange at all times.
- **Conciseness:** The clarity and brevity of the codebase, making it easier to understand and maintain. Smart Eye follows a modular structure where each class has a single well-defined responsibility: DetectionEngine handles inference, ByteTrack manages track state, IDMappingManager handles Re-ID, and DatabaseManager handles all persistence operations.
- **Maintainability:** The simplicity with which Smart Eye may be updated, corrected, or extended during the course of its operational life. Configuration parameters are

externalised to a JSON file, YOLO models are swappable at runtime, and the SQLite schema is versioned, all of which reduce the cost of future modifications.

- **Documentation:** The presence and quality of documentation that describes Smart Eye's functionality, design, and usage. This practicum report serves as the primary documentation artefact, supplemented by inline docstrings in the source code and a user-facing configuration reference embedded in the Settings tab tooltips.
- **Legibility:** The readability and clarity of the source code, making it easier for developers to understand and work with. Smart Eye uses descriptive variable and function names, consistent indentation, and PEP-8 compliant Python style throughout the codebase.

External Quality:

- **Features:** The functionality and capabilities of Smart Eye that meet operator needs. The system provides real-time vehicle detection and tracking, automatic wash session logging with entry and exit snapshots, configurable monitoring zones, overtime and vacuum full alerts, session history with filtering and export, and an analytics dashboard with charts and heatmaps.
- **Speed:** The responsiveness and performance of Smart Eye in terms of executing tasks and processing data. The video processing pipeline is optimised through configurable frame skipping, dynamic confidence thresholds, and background model loading, ensuring the GUI remains responsive while detection and tracking run at the configured frame rate.
- **Space:** The efficiency of memory usage and storage requirements. Smart Eye stores session snapshots as compressed JPEG images and uses SQLite WAL mode for efficient, append-friendly database writes. The configuration state is held in a lightweight JSON file, keeping the total disk footprint proportional only to the number of recorded sessions.
- **Network Usage:** The efficiency and effectiveness of the system's network communication. When operating with an RTSP camera stream, Smart Eye connects directly to the stream URL configured by the operator and reads frames using OpenCV's native RTSP support, with no additional network services, cloud dependencies, or background data transfers.

- **Stability:** The capacity of Smart Eye to function without errors or failures over extended operating periods. The system handles video source disconnections with automatic reconnection attempts, uses SQLite WAL mode to ensure crash-safe writes, and isolates the detection thread so that a pipeline exception does not crash the GUI.
- **Robustness:** The ability of Smart Eye to handle unexpected or erroneous inputs without failure. The system validates all configuration parameters before saving, applies low-light confidence adjustments when frame brightness drops below a configurable threshold, and gracefully handles missing model files or invalid video sources by displaying informative error messages to the operator.
- **Ease-of-Use:** The user-friendliness and intuitiveness of Smart Eye's interface. The PyQt5 desktop application is organised into four clearly labelled tabs, presents all real-time metrics on a single screen, and requires no technical knowledge to operate. Detection is started with a single button click, and monitoring zones are drawn interactively using mouse drag gestures.
- **Determinism:** The predictability and consistency of Smart Eye's behaviour given the same inputs. With identical video input and configuration parameters, the detection and tracking pipeline produces consistent bounding box assignments, Re-ID matches, and session log entries, ensuring that operator-reported observations can be reliably reproduced for debugging and verification purposes.
- **Back-Compatibility:** The ability of Smart Eye to work with previous configuration files and database records when the application is updated. The JSON configuration loader applies default values for any missing keys introduced in newer versions, and the SQLite schema uses additive migrations so that existing session records remain accessible after an update.

These qualities, both internal and external, contribute to the overall quality of Smart Eye, ensuring that the system meets the desired operational standards, functions correctly under real-world car wash conditions, and satisfies the monitoring and reporting requirements of the operator.

Chapter 9: Conclusion

In the present era of modern technology, the efficient management of operational processes is crucial for small businesses seeking to remain competitive and deliver consistent service quality. It is a privilege to have had the opportunity to develop a system that directly addresses a real operational challenge faced by car wash facilities, specifically the lack of automated, reliable monitoring of vehicle activity and service durations.

The practicum programme provided an invaluable platform for applying the theoretical knowledge acquired throughout undergraduate engineering studies to a concrete, deployable software product. The experience of designing, implementing, and documenting Smart Eye from inception to completion has deepened the understanding of how academic disciplines such as computer vision, software engineering, database design, and human-computer interaction converge in a real-world development context.

I am truly grateful for the guidance and support received throughout this practicum, which made it possible to navigate the technical challenges of building a production-quality system and to document the process with the rigour expected of a professional software engineering report.

9.1 Practicum and Its Value

The main objective of this report is to describe a system that enhances the efficiency and accuracy of vehicle monitoring at car wash facilities by replacing manual observation with an automated, computer-vision-driven solution. Smart Eye eliminates the need for staff to manually count vehicles, time wash durations, or monitor vacuum bay occupancy, tasks that are error-prone, labour-intensive, and impractical to sustain continuously during busy operating hours.

By adopting Smart Eye, a car wash facility gains real-time visibility of all vehicle activity, automatic session records with entry and exit timestamps and snapshot images, configurable alerts for overtime and vacuum full conditions, and access to historical analytics that reveal daily throughput, average wash duration, and peak operating hours. These

capabilities make the monitoring process more effective and efficient, requiring minimal operator intervention during normal operation.

Undergraduate engineering studies provide students with both theoretical and practical knowledge. The practicum programme offers an opportunity to apply this knowledge and gain practical working skills by developing and observing real-world operational systems. The development of Smart Eye was a successful and enriching experience that deepened understanding of object detection with YOLO11 (Redmon et al., 2016; Jocher, Chaurasia and Qiu, 2024), multi-object tracking with ByteTrack (Zhang et al., 2022), appearance-based Re-Identification (Ye et al., 2022), PyQt5 desktop application architecture, and SQLite database management (Riverbed Technology, no date).

Practical work experience holds immense value as it provides a unique perspective that cannot be obtained through theoretical learning alone. Prior to entering the job market, students should strive to gain real-world development experience in their field of study. The skills and insights gained through this practicum, including system analysis, risk management, software design, and technical documentation, will significantly enhance employability and professional readiness in the field of software and computer vision engineering.

9.2 Conclusion

In summary, Smart Eye stands as a comprehensive automated monitoring solution for car wash facilities, harnessing the capabilities of YOLO11 object detection (Redmon et al., 2016; Jocher, Chaurasia and Qiu, 2024), ByteTrack multi-object tracking (Zhang et al., 2022), and appearance-based Re-Identification (Ye et al., 2022) to deliver accurate, persistent vehicle tracking across all monitored zones. The system harmonises real-time video processing with a responsive PyQt5 desktop interface, an SQLite session database, and a configurable JSON settings layer, creating a cohesive platform that serves the practical needs of the car wash operator.

Smart Eye successfully achieves all core objectives established at the outset of the project: vehicles are detected and tracked in real time without manual intervention; wash sessions are logged automatically with entry and exit timestamps, duration, status, and snapshot images; overtime and vacuum full alerts are raised immediately when configurable thresholds are breached; and the operator has full access to session history, filtering, export, and analytics through an intuitive four-tab desktop interface.

The system's three-tier layered architecture ensures that the GUI, the processing pipeline, and the data layer remain decoupled, a design decision that promotes maintainability, testability, and the ability to extend the system with new capabilities in future versions. The use of SQLite WAL mode, Qt signal-based inter-thread communication, and runtime model switching further demonstrates that Smart Eye was designed not only to function correctly but to operate reliably under the continuous, unattended conditions of a real car wash environment.

Overall, the development and documentation of Smart Eye through this practicum programme reflects the successful integration of computer vision, software engineering, and operational design principles into a practical, deployable product that addresses a genuine need in the car wash industry.

9.3 Future Plan

As Smart Eye represents the initial version of the system, there are plans to enhance it further by incorporating additional features based on operator feedback and evolving operational requirements. The goal is to continuously improve and extend the system while maintaining the stability and usability of the current release. Some of the potential future enhancements that can be considered are:

- Multi-camera support: extending the monitoring pipeline to handle multiple simultaneous video streams, allowing a single installation to cover the full extent of a large car wash facility with several wash bays and vacuum areas.

- Cloud-based session storage and remote dashboard: synchronising session records and analytics to a cloud backend, enabling the car wash owner to monitor facility performance remotely from a web or mobile dashboard.
- License plate recognition integration: augmenting the vehicle tracking pipeline with an OCR-based license plate reader to provide unique vehicle identification independent of the tracker ID, enabling customer loyalty programmes and repeat-visit analytics.
- Mobile push notifications for overtime alerts: delivering overtime and vacuum full alerts directly to the operator's mobile device so that intervention can occur even when the operator is away from the monitoring workstation.
- AI-based wash quality assessment: incorporating a secondary classification model that analyses vehicle appearance before and after the wash cycle to provide an automated quality score, supporting service consistency and customer satisfaction tracking.

References

- [1] Bewley, A., Ge, Z., Ott, L., Ramos, F. and Upcroft, B. (2016) 'Simple online and realtime tracking', IEEE International Conference on Image Processing (ICIP). Phoenix, USA. IEEE, pp. 3464–3468.
- [2] Booch, G., Rumbaugh, J. and Jacobson, I. (2005) The unified modeling language user guide. 2nd edn. Boston: Addison-Wesley Professional.
- [3] Bradski, G. and Kaehler, A. (2008) Learning OpenCV: computer vision with the OpenCV library. Sebastopol, CA: O'Reilly Media.
- [4] Charette, R.N. (1989) Software engineering risk analysis and management. New York: McGraw-Hill.
- [5] Date, C.J. (2003) An introduction to database systems. 8th edn. Boston: Pearson Education.
- [6] IFPUG (2010) Function point counting practices manual. Release 4.3.1. Westerville, OH: International Function Point Users Group.
- [7] Jocher, G., Chaurasia, A. and Qiu, J. (2024) Ultralytics YOLO (Version 11.0) [Computer program]. Available at: <https://github.com/ultralytics/ultralytics> (Accessed: 10 May 2026).
- [8] Pressman, R.S. and Maxim, B.R. (2014) Software engineering: a practitioner's approach. 8th edn. New York: McGraw-Hill Education.
- [9] Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2016) 'You only look once: unified, real-time object detection', Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). Las Vegas, USA. IEEE, pp. 779–788.
- [10] Riverbed Technology (no date) 'SQLite WAL mode and concurrent access', Technical Reference Guide. Available at: <https://www.sqlite.org/wal.html> (Accessed: 10 May 2026).

- [11] Rivertz, H.J. (2021) 'Real-time vehicle detection and counting using deep neural networks for traffic monitoring applications', *Journal of Visual Communication and Image Representation*, 75.
- [12] Sommerville, I. (2016) *Software engineering*. 10th edn. Harlow: Pearson Education Limited.
- [13] Ye, M., Shen, J., Lin, G., Xiang, T., Shao, L. and Hoi, S.C.H. (2022) 'Deep learning for person re-identification: a survey and outlook', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(6), pp. 2872–2893.
- [14] Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W. and Wang, X. (2022) 'ByteTrack: multi-object tracking by associating every detection box', *European Conference on Computer Vision (ECCV)*. Tel Aviv, Israel. Springer, pp. 1–21.